Étudiant	num	nom	adresse	age
	1	Bélaïd	Maisel	20
	2	Millot	CROUS	20
	3	Silber	Maisel	21

UV	codeUV	nb H	coord
	IO	45	**Pierre**
	BD	21	**Paul**

Inscrit	numÉtudiant	codeUV	note
	2	BD	10
	1	BD	20
	2	IO	17
	3	IO	18

II Le modèle relationnel

1 Concept

Clé

- Une **clé** est un groupe minimum d'attributs qui détermine un n-uplet unique dans une relation (à tout instant)

Exemple

- Clé de Étudiant ?
- Clé de UV ?
- Clé de Inscrit ?

II Le modèle relationnel

1 Concept

<u>Clé</u>

Contrainte d'intégrité

- Toute **relation doit posséder une clé** renseignée (sans valeur inconnue)

II Le modèle relationnel

1 Concept

<u>Schéma d'une relation</u>

Le schéma d'une relation décrit :

- Son nom
- La liste des attributs qu'elle comporte et des domaines associés
- La liste des attributs composant la clé (la clé est soulignée)

Exemple

•Étudiant(num : entier, nom : chaîne, adresse : chaîne, age : entier de 18 à 35)

II Le modèle relationnel

1 Concept

Schéma d’une relation

Intention vs. Extension

- Schéma de relation : intention de la relation
- Table : extension
- Schéma d'une BD relationnelle : ensemble des schémas des relations

II Le modèle relationnel

1 Concept

<u>Clé étrangère</u>

Une ***clé étrangère*** est un groupe d'attributs qui apparaît comme clé dans une autre relation

- R1(A1, A2, ... , **Ap, Ap+1**, ..., An)
- R2(**B1, B2**, ..., Bn)

Rôle

Les clés étrangères définissent des **contraintes d'intégrité référentielle** entre relations

Étudiant	num	nom	adresse	age
	1	Bélaïd	Maisel	20
	2	Millot	CROUS	20
	3	Silber	Maisel	21

UV	codeUV	nb H	coord
	IO	45	**Pierre**
	BD	21	**Paul**

Inscrit	numÉtudiant	codeUV	note
	2	BD	10
	1	BD	20
	2	IO	17
	3	IO	18

II Le modèle relationnel

1 Concept

Clé étrangère

- **Mises à jour et clés étrangères** Insertion : la valeur des attributs doit exister dans la relation référencée.
  - Insertion de (4, ‘BD’, 15) dans Inscrit ?
- Suppression dans la relation référencée; les n-uplets référençant doivent disparaître.
  - Suppression de l’étudiant 2 dans Étudiant ?

II Algèbre relationnelle

Définition

2 types de langage associés au modèle relationnel:

- Langages de Définition de données (LDD)
  - Définition/mise à jour des schémas de relations
- Langages de manipulation de données (LMD)
  - Intérogation, ← Op. de AR
  - mise à jour (suppr. Insertion, modif.)

II Algèbre relationnelle

Exemple

Schéma de base de données

Etudiant(num,nom,adresse,age)

UV(codeUV,nbH,coord)

Inscrit(numEtudiant, codeUV,node)

II Algèbre relationnelle

Principes des Opérateurs

Tout résultat d'un opérateur est une relation

R -> Op -> R'

Opérateurs unaires et binaires

- **Unaires:** restriction (sélection), projection
- **Binaires** : union, intersection, produit cartésien , jointure, divison.

II Algèbre relationnelle

Restriction

Permet de "sélectionner" des tuples

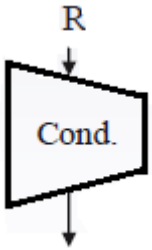
Réduit la taille de la relation verticalement

Contraintes

- Unaire
- Spécifier une condition

Notation

- Notation textuelle : T $\sigma_{cond}(R)$
- Notation graphique :



II Algèbre relationnelle

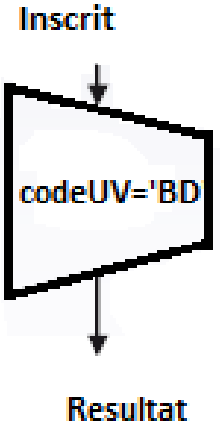
Restriction

Exemple : **Quelles sont les inscriptions au cours Bases de Données ?**

Etudiant(num,nom,adresse,age)

UV(codeUV,nbH,coord)

Inscrit(numEtudiant, codeUV,node)



Result <- $\sigma_{codeuv='BD'}$(Inscrit)

II Algèbre relationnelle

Restriction

Exemple : **Quelles sont les inscriptions au cours de Bases de Données ?**

Inscrit	numÉtudiant	codeUV	note
	2	BD	10
	1	BD	20
	2	IO	17
	3	IO	18

Exemple: **Quels sont les bons étudiants (note > = 15) inscrits au cours de Bases de Données ?**

II Algèbre relationnelle

Projection

Permet de "sélectionner" des attributs

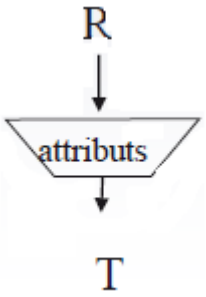
La projection réduit la taille de la relation horizontalement

Contraintes

- Unaire
- Spécifier une liste d'attributs

Notation

- textuelle : $T \leftarrow \pi_{attributs}(R)$
- graphique



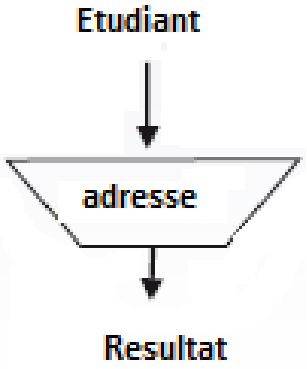
II Algèbre relationnelle

Projection

Quelles sont les adresses des étudiants ?

Etudiant(num,nom,adresse,age)

Result <- $\pi_{adresse}$(Etudiant)



Étudiant	num	nom	adresse	age
	1	Bélaïd	Maisel	20
	2	Millot	CROUS	20
	3	Silber	Maisel	21

II Algèbre relationnelle

Projection

Quels sont les codes et les nombres d'heures des Uvs ?

UV(codeUV,nbH,coord)

II Algèbre relationnelle

Projection et restriction

Quels sont les bons étudiants (note > = 15) inscrits en Bases de Données ? (donner les numéros des étudiants).

II Algèbre relationnelle

Projection et restriction

Méthode :

- Analyse de la question pour identifier les relations et attributs en jeu
  - Limiter le nombre de relations en jeu
- Analyse de la question pour identifier les opérateurs en jeu

  -Quels sont les ..? => projection

  -Qui respectent une condition => restriction

II Algèbre relationnelle

Produit cartesien

But

•Ensemble de tous les tuples obtenus par concaténation de chaque tuple de R avec chaque tuple de S

Contraintes

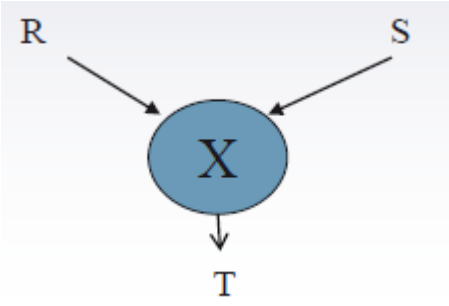
Binaire

Schéma du résultat:

- R(a1, a2, ..., an), S(b1, b2, ..., bp)
- T <- R X S, T(a1, a2, ..., an, b1, b2, ..., bp)
- Card (R X S) = Card (R) * Card (S)

Notation

- Notation textuelle : T <- R X S
- Notation graphique :



II Algèbre relationnelle

Produit cartésien

Étudiant	num	nom	adresse	age
	1	Bélaïd	Maisel	20
	2	Millot	CROUS	20
	3	Silber	Maisel	21

UV	code	nbH	coord
	IO	45	Conan
	BD	21	Lecocq

Étudiant X UV	num	nom	adresse	age	code	nbH	coord
	1	Bélaïd	Maisel	20	IO	45	Conan
	2	Millot	CROUS	20	IO	45	Conan
	3	Silber	Maisel	21	IO	45	Conan
	1	Bélaïd	Maisel	20	BD	21	Lecocq
	2	Millot	CROUS	20	BD	21	Lecocq
	3	Silber	Maisel	21	BD	21	Lecocq

II Algèbre relationnelle

jointure

But

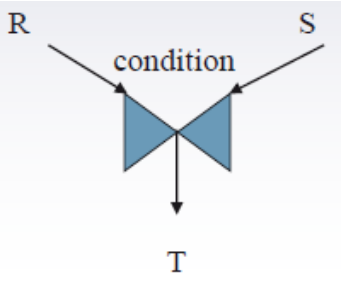
Permet d'établir le lien **sémantique** entre les relations

Contraintes

- Binaire
- Schéma du résultat :

-R(a1, a2, ..., an), S(b1, b2, ..., bp)

-T <- $R \underset{\text{condition}}{\bowtie} S$, T(a1, a2, ..., an, b1, b2, ..., bp)



Notation

- textuelle : T <- $R \underset{\text{condition}}{\bowtie} S$
- Graphique

II Algèbre relationnelle

jointure

Étudiant	num	nom	adresse	age
	1	Bélaïd	Maisel	20
	2	Millot	CROUS	20
	3	Silber	Maisel	21

Étudiant.num=Chambre.numÉtudiant

Chambre	no	prix	numÉtudiant
	10	200	3
	21	150	2

Étudiant $\bowtie_{\text{num=numÉtudiant}}$ Chambre	num	nom	adresse	age	no	prix	numÉtudiant
	2	Millot	CROUS	20	21	150	2
	3	Silber	Maisel	21	10	200	3

II Algèbre relationnelle

jointure

Inscrit	numÉtudiant	codeUV	note
	2	BD	10
	1	BD	20
	2	IO	17
	3	IO	18

Inscrit.numÉtudiant=Étudiant.num

Étudiant	num	nom	adresse	age
	1	Bélaïd	Maisel	20
	2	Millot	CROUS	20
	3	Silber	Maisel	21

Inscrit ⋈ Étudiant (numÉtudiant=num)	num	nom	adresse	age	numÉtudiant	codeUV	note
	1	Bélaïd	Maisel	20	1	BD	20
	2	Millot	CROUS	20	2	IO	17
	2	Millot	CROUS	20	2	BD	10
	3	Silber	Maisel	21	3	IO	18

II Algèbre relationnelle

Opérateur U (ensembliste)

But

- Permet de fusionner 2 relations

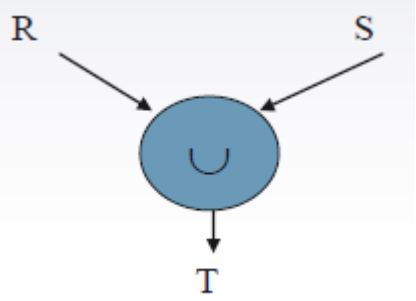
Contraintes

- Binaire
- **Même schéma**

Notation

- Notation textuelle : T <- R U S
- Notation graphique :

L'opérateur Union fait parti des plus simples et moins utilisés.



II Algèbre relationnelle

Opérateur U (ensembliste)

- Quels sont les noms des personnes (professeurs et étudiants) habilitées à entrer sur le campus ?
- Utiliser la projection pour avoir les mêmes attributs

Étudiant	num	nom	adresse	age
	1	Bélaïd	Maisel	20
	2	Millot	CROUS	20
	3	Silber	Maisel	21

Enseignant	num	nom	bureau	Titre
	10	Conan	B302	Maitre de conférence
	20	Lecocq	D307	Maitre de conférence
	3	Millot	B305	Doctorant

II Algèbre relationnelle

Opérateur ∩ (ensembliste)

But

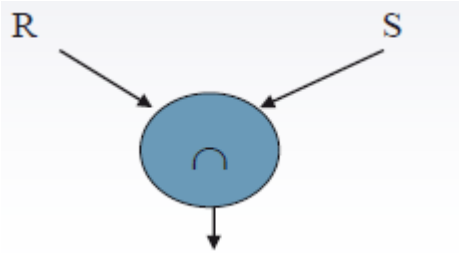
- Permet d'obtenir l'ensemble des tuples appartenant à deux relations

Contraintes

- Binaire
- **Même schéma**

Notation

- Notation textuelle : T<- R ∩S
- Notation graphique :



II Algèbre relationnelle

Opérateur ∩ (ensembliste)

- Quels sont les noms portés par des professeurs et des étudiants ?

Étudiant	num	nom	adresse	age
	1	Bélaïd	Maisel	20
	2	Millot	CROUS	20
	3	Silber	Maisel	21

Enseignant	num	nom	bureau	Titre
	10	Conan	B302	Maitre de conférence
	20	Lecocq	D307	Maitre de conférence
	3	Millot	B305	Doctorant

II Algèbre relationnelle

Opérateur – (différence)

But

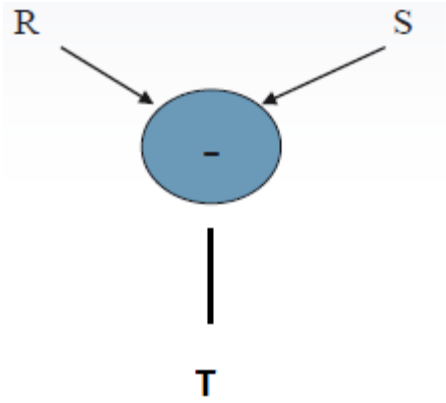
Obtenir l'ensemble des tuples d'une relation qui ne figurent pas dans une autre

Contraintes

- Binaire
- **Même schéma**
- Non commutatif

Notation

- Notation textuelle : T = R -S
- Notation graphique :



II Algèbre relationnelle

Opérateur – (différence)

Quels sont les noms des étudiants qui NE sont PAS des noms de professeurs ?

Étudiant	num	nom	adresse	age
	1	Bélaïd	Maisel	20
	2	Millot	CROUS	20
	3	Silber	Maisel	21

Enseignant	num	nom	bureau	Titre
	10	Conan	B302	Maitre de conférence
	20	Lecocq	D307	Maitre de conférence
	3	Millot	B305	Doctorant

II Algèbre relationnelle

Synthèse

Opérateur	Sémantique	Notation textuelle	Notation graphique
Restriction	« Sélectionner » des tuples	$T \leftarrow \sigma_{cond}(R)$	Cond.
Projection	« Sélectionner » des attributs	$T \leftarrow \Pi_{attributs}(R)$	attributs
Union	Fusionner les extensions de 2 relations	$T \leftarrow R \cup S$	$\cup$
Intersection	Obtenir l'ensemble des tuples communs à deux relations	$T \leftarrow R \cap S$	$\cap$
Différence	Tuples d'une relation qui ne figurent pas dans une autre	$T \leftarrow R - S$	$-$
Produit cartésien	Concaténer chaque tuple de R avec chaque tuple de S	$T \leftarrow R \times S$	X
Jointure	Etablir le lien sémantique entre les relations	$T \leftarrow R \bowtie_{condition} S$	$\bowtie$

Exemple de requête

Base de donnée : Vins

- Vins(num, cru, annee, degre)
- Recoltes(nvin, nprod, quantite)
- Producteurs(num, nom, prenom, region)
- Clients(num, nom, prenom, ville)
- Commandes(ncde, date, ncli, nvin, qte)
- Livraisons(ncde, no_ordre, qteLivree)

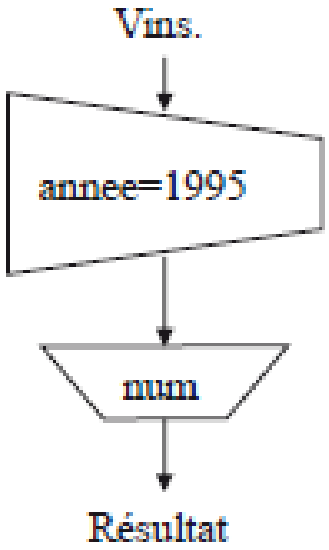
Exemple de requête

Requête

- **Temp** = $\sigma_{annee=1995}$ (Vins)
- Resultat = π_{num}(**Temp**)

  oubien

- Resultat = $\pi_{num}(\sigma_{annee=1995}(Vins))$



A quelle requête correspond la réponse?

Exemple de requête

Requête

Quels sont les noms des producteurs de Muscadet (cru)?

Démarche

1. Identifier les attributs impliqués:
2. Identifier les opérateurs relationnels dont on aura besoin.

Exemple de requête

Requête

Quels sont les noms des producteurs de Muscadet (cru)?

- Vins(num, cru, annee, degre) -> Cru
- Producteurs(num, nom, prenom, region) ->nom des pruduction
- Recoltes(nvin, nprod, quantite) -> lien

Exemple de requête

Requête

Quels sont les noms des producteurs de Muscadet (cru)?

Muscadet (restriction)

Noms (projection)

Lien(jointure)

$$\pi_{nom}\ \text{producteurs} \bowtie_{num=nprod} \text{Recoltes} \bowtie_{nvin=num} \sigma_{cru=muscadet}(\text{vins})$$

Exemple de requête

Requête

Quels sont les numéros des vins ne faisant l’objet d’aucune commande.

Différence

II Algèbre relationnelle

Optimisation des requêtes

Requêtes

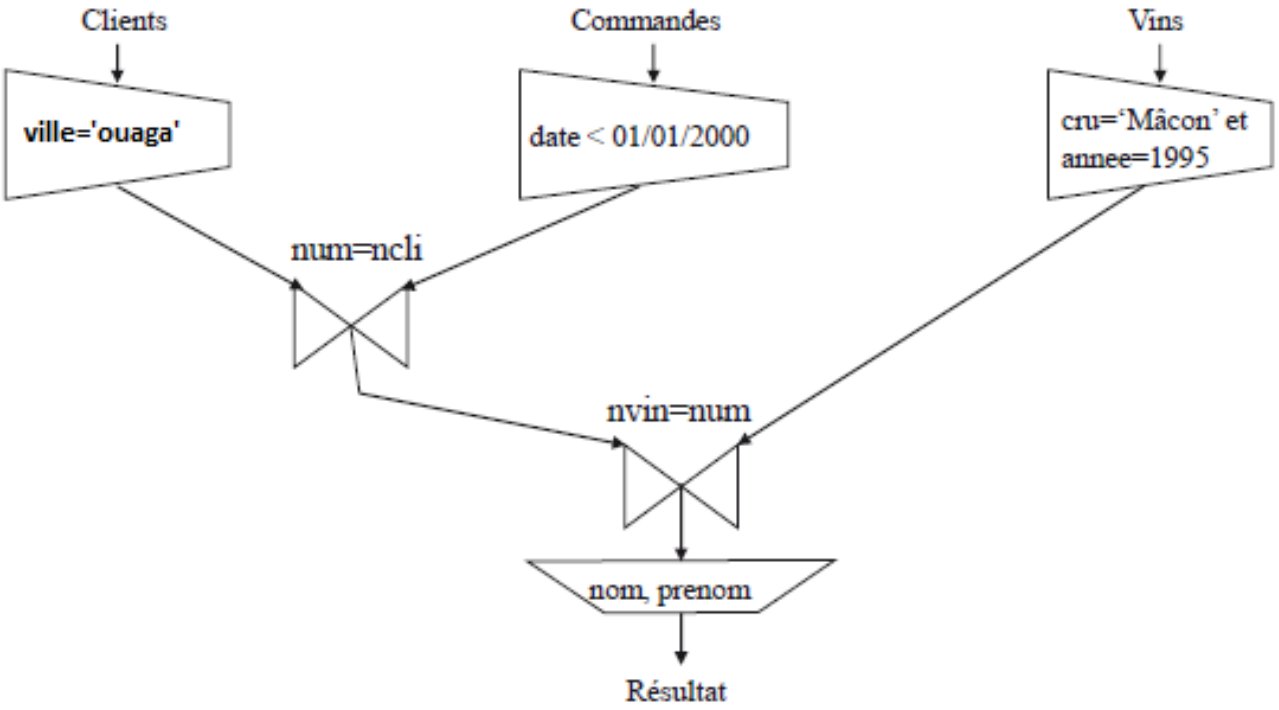
Noms et prénoms des clients habitant ouaga ayant commandé du Mâcon 1995 avant le 01 janvier 2000

- Plusieurs réponses possibles
- Optimiser = choisir la meilleure combinaison
  - Les bons opérateurs
  - Dans le meilleur ordre

II Algèbre relationnelle

Optimisation des requêtes

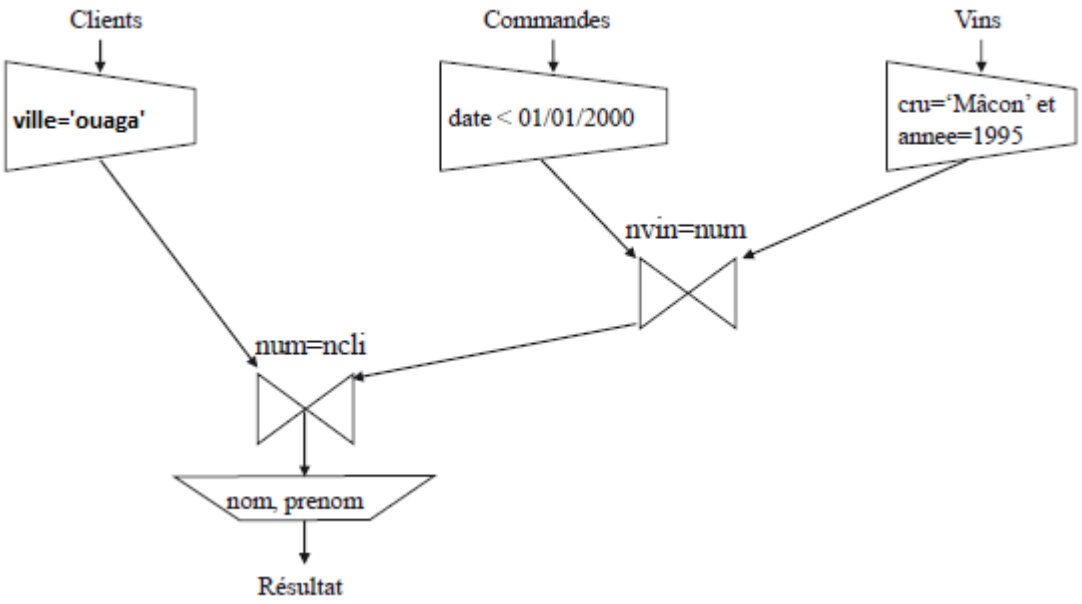
Réponse 1



II Algèbre relationnelle

Optimisation des requêtes

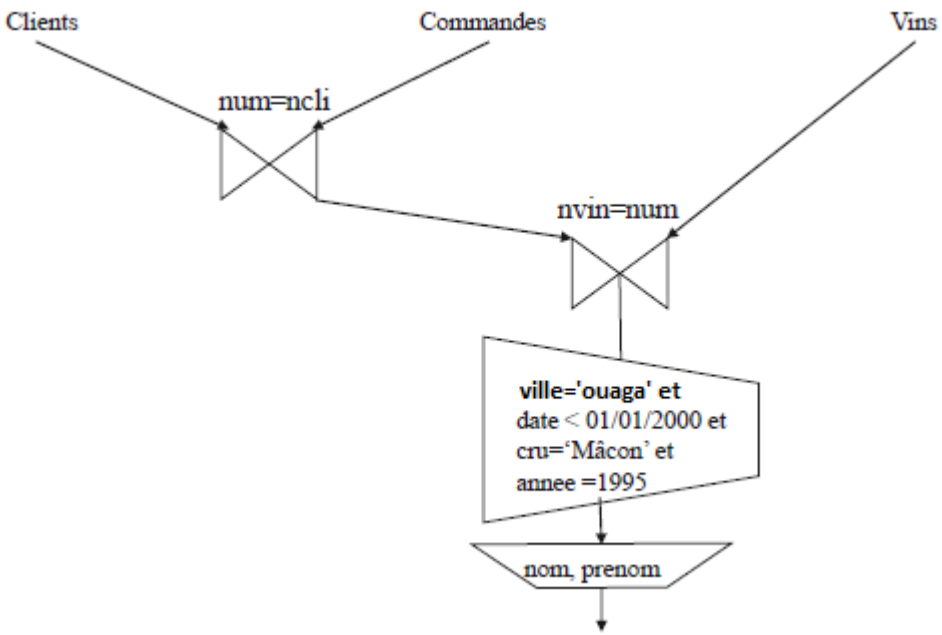
Réponse 2 (ordre des jointures)



II Algèbre relationnelle

Optimisation des requêtes

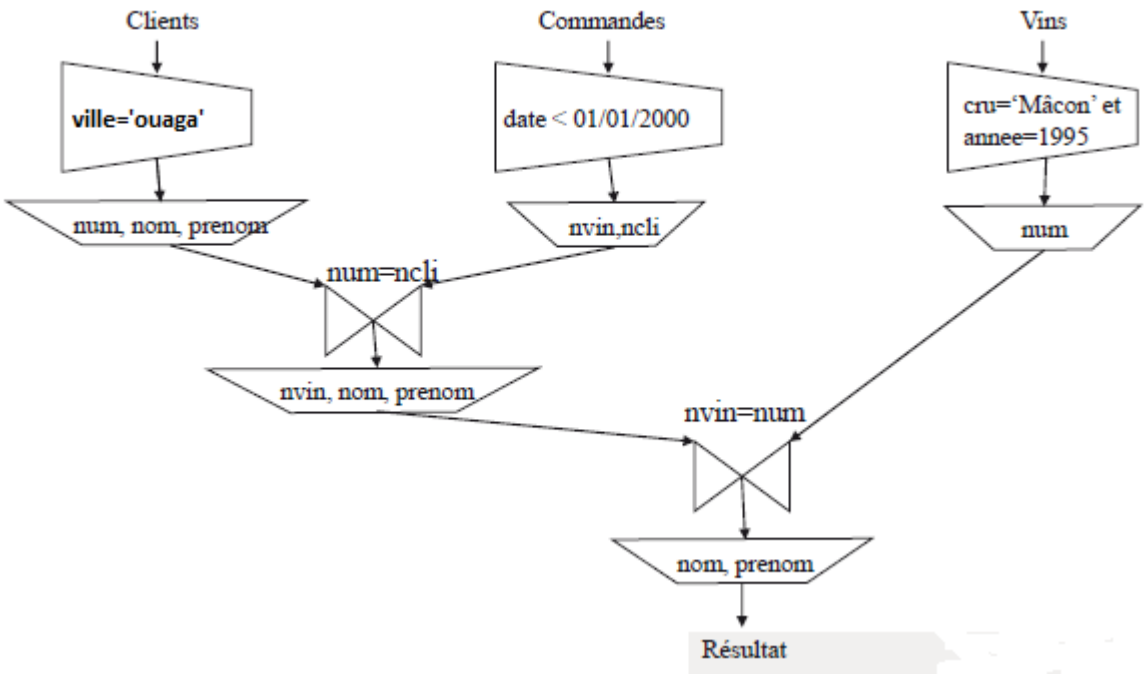
Réponse 3



II Algèbre relationnelle

Optimisation des requêtes

Réponse 3



II Algèbre relationnelle

Optimisation des requêtes

Heuristique la plus employée

- Remonter les opérateurs unaires (restriction, projection) => diminution de la taille des relations
- Descendre les jointures

II Algèbre relationnelle

Optimisation des requêtes

Facteurs déterminants

- Ordre d'exécution des opérations algébriques
- Algorithme implantant de façon optimale les opérations algébriques.
- Placement des données sur le disque
- Taille des relations intermédiaires
- Statistiques d’exécution de ces requêtes

….

Responsabilité du SGBD

II Algèbre relationnelle

Exercices

III – Conception

Exemple introductif

Cas du carnet d’adresses.

III – Conception

Introduction

1. Schémas normalisés

Un schéma normalisé présente des caractéristiques formelles et des règles qui nous indiquent plus précisément les liens qui caractérisent les données.

La **normalisation** nous garantit l'absence de défaut (notamment de redondance) tout en préservant l'intégralité de l'information représentée.

III – Conception

Introduction

1. Schémas normalisés

L'objectif de la normalisation est de construire un schéma de base de données cohérent.

Un modèle relationnel normalisé doit respecté certaines contraintes appelées **formes normales.**

Les formes normales se basent sur les **dépendances fonctionnelles** entre attributs.

III – Conception

Introduction

2. Dépendances fonctionnelles

Définition

Soit R une relation, A et S des attributs de R.

On dit que A dépend fonctionnellement de S, (notée $S \rightarrow A$) Si une valeur de l'attribut de S induit une unique valeur de A. Autrement dit

Si $t_1[S]=t_2[S]$ alors $t_1[A] = t_2[A]$

Si je connais S alors je connais A

III – Conception

Introduction

2. Dépendances fonctionnelles

Exemple

Etudiant(num,nom,adresse,age)

num -> nom : est une dépendance fonctionnelle.

nom -> num n’est pas une dépendance fonctionnelle.

III – Conception

Introduction

2. Dépendances fonctionnelles

Transitivité

S->A et A-> T alors S -> T

Exemple

numCommande -> NumClient -> NomClient

On a alors NumCommande -> nomClient.

III – Conception

Introduction

2. Dépendances fonctionnelles

Définition

Une clé (d’une relation R) est un sous-ensemble minimal C des attributs tel que tout attribut de R dépend fonctionnellement de C.

III – Conception

Introduction

3. Normalisation

Première forme normale (1FN)

Une relation est en 1FN si tous ses attributs sont atomiques (mono-valués c'est-à-dire une valeur par ligne). Aucun attribut n'est décomposable en plusieurs attributs.

Elle possède une clé identifiant de manière unique chaque tuple.

III – Conception

Introduction

3. Normalisation

Première forme normale (1FN)

Employe(nom, diplôme)

NOM	DIPLOMES	
	Nature	Année
DUPONT	Bac	1975
	Licence	1980

III – Conception

3. Normalisation

Deuxième forme normale (2FN)

Une relation est en 2 FN si et seulement si:

- Elle est 1FN
- Tout attribut n’appartenant pas à la clé primaire est en dépendance totale avec la clé primaire (Cas de clé primaire composée de plusieurs attributs)

III – Conception

3. Normalisation

Deuxième forme normale (2FN)

Exemple

Commande(numCde, refProd, designProd, Quantité)

1FN = OK,

2FN = NON, car refProd $\longrightarrow$ designProd , pas besoin de connaitre numCde.

Commande(numCde,RefProd,Quantité)

Produit(refProd, designProd)

III – Conception

3. Normalisation

Deuxième forme normale (2FN)

- Séparation des attributs concernés par la dépendance dans une autre relation.

- Une relation en 1 NF dont la clé est mono-attribut est une 2 NF.

III – Conception

3. Normalisation

Deuxième forme normale (2FN)

Exemple

Enseignement(num, codeMat, nomEnseignant, volumeHoraire).

Avec num $\longrightarrow$ NomEnseignant

Enseignement(num, codeMat, volumeHoraire).

Enseignant(num,nomEnseignant)

III – Conception

3. Normalisation

Troisième forme normale (3FN)

Une relation est en 3° forme normale si et seulement si

- Elle est en 2FN,
- Elle ne contient pas de dépendance fonctionnelle entre attributs non clé primaire.

III – Conception

3. Normalisation

Troisième forme normale (3FN)

Exemple

Enseignant(num, nom, catégorie, classe, salaire)

Catégorie, classe -> salaire;

- **Isoler la DF transitive dans une nouvelle relation**

Enseignant(num,nom,categorie,classe)

Salaire(categorie, classe, salaireEns).

III – Conception

Le Modèle Entité/Association

Proposé par Chen en 1976

- Simple
- Graphique
- Facilite le dialogue avec

les utilisateurs (concepteurs <->clients)

- Utilisé dans la plupart des méthodes de Conception.

The entity-relationship model—toward a unified view of data
ACM Transactions
On Database Systems. 1976

III – Conception

A - Le Modèle Conceptuel des Données (MCD)

Le modèle Entité/Association (E/A) propose essentiellement une notation pour soutenir la démarche de conception de schéma présentée précédemment.

La notation E/A a pour caractéristiques d'être simple et suffisamment pour modéliser des structures relationnelles. De plus, elle repose sur une représentation graphique qui facilite considérablement sa compréhension.

Un inconvénient du modèle E/A est de mener à certaines ambiguïtés pour des schémas complexes.

III – Conception

A - Le Modèle Conceptuel des Données (MCD)

- Le modèle conceptuel des données (MCD) a pour but de représenter de façon structurée les données qui seront utilisées par le SI.
- Le modèle conceptuel des données décrit la sémantique c’est à dire le sens attaché à ces données et à leurs rapports (non à l’utilisation qui peut en être faite.)

III – Conception

A - Le Modèle Conceptuel des Données (MCD)

- On établit le MCD après avoir recensé et donné un nom à l'ensemble des données du domaine étudié.
- Ensuite on étudie les relations existantes entre ces données, pour aboutir au MCD.

A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

Le développement du MCD se fait par itérations

Quand on a stabilisé le MCD et qu'on a fait le choix du SGBD pour le développement du SI, on peut alors passer au MLD.

Dans ce cours nous nous intéressons à un formalisme de modélisation conceptuelle des données appelé **Modèle Entité Association.**

A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

Il existe 4 concepts de base dans le formalisme entité-association.

a) Entité
b) Association
c) Propriété ou attribut
d) Cardinalité.

A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

a-Entité

Un objet concret ou abstrait du monde réel pour lequel nous voulons conserver de l'information (objet pertinent dans le contexte du SI étudié).

Une entité peut être:

- Une personne (employé, étudiant, client,…)
- Un endroit (bureau, ville, territoire, pays,…)
- Un événement (commande, livraison, …)
- Etc.

A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

a-Entité

Les entités d'un MCD dépendent de l'application pour laquelle le SI est développé.

Une entité représente un ensemble d'objets ayant les mêmes caractéristiques et non pas une occurrence de ces objets.

Exemple:

Entité ETUDIANT,

Occurrence: Michel Paré

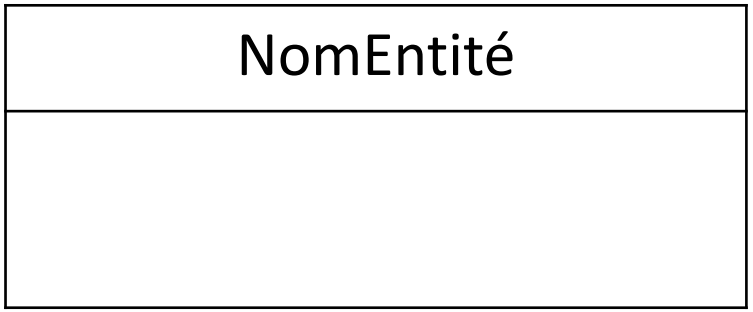
A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

a-Entité

Une entité est nommée en utilisant un nom.

Une entité est représentée de la façon suivante.



A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

b-Association entre les entités

Une association représente une relation générique qui existe entre deux (ou plusieurs) entités.

Il est courant de nommer les associations en utilisant des verbes.

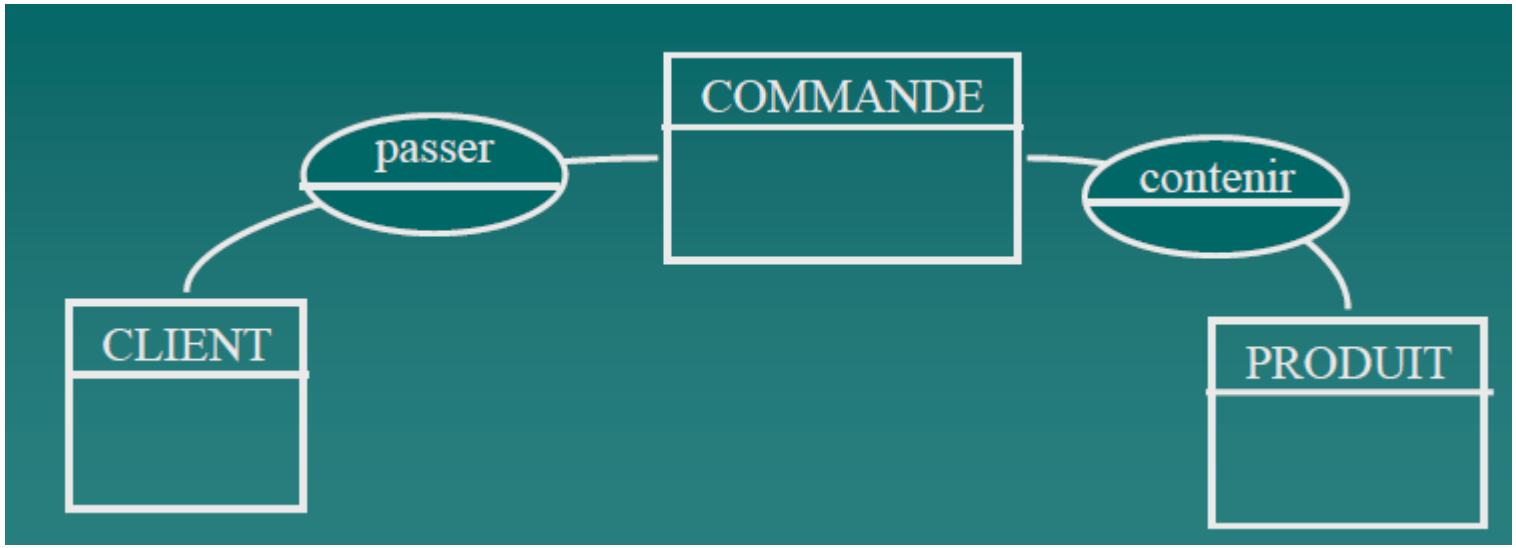
A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

b-Association entre les entités

Exemple : Un CLIENT passe une COMMANDE,

Une COMMANDE contient des PRODUITS.



A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

c-Propriétés ou attributs d'une entité

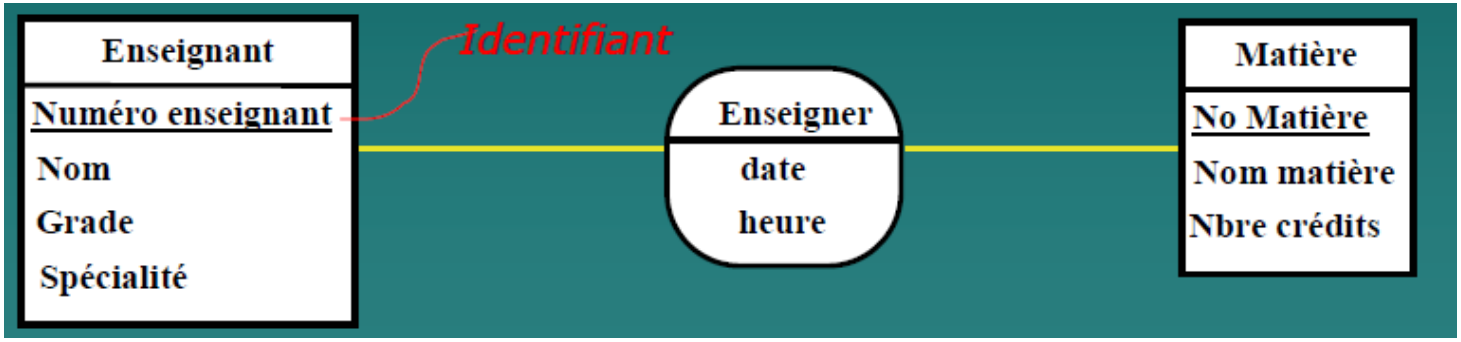
- Un attribut est donnée élémentaire utilisée pour décrire une entité ou une association.
- Un attribut recevra une valeur précise pour chaque occurrence de l'entité ou de l'association.
- Un ou plusieurs attributs peuvent jouer le rôle d'identifiant de l'entité (cela deviendra la clé d'une table au niveau MLD)

A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

c-Propriétés d'une entité

L'identifiant d'une entité est une propriété ou un ensemble de propriétés qui permettent de repérer une occurrence de l'entité de façon unique.



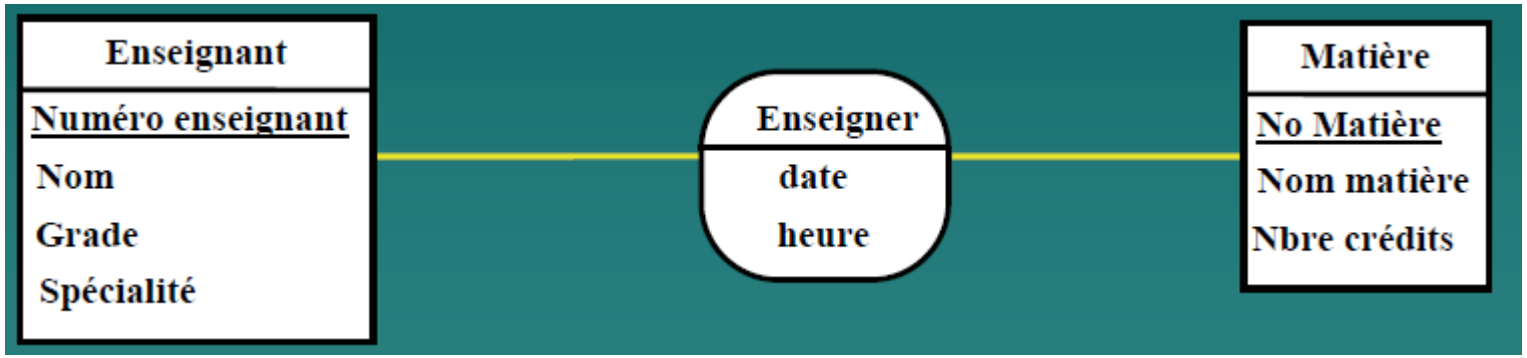
! Toute entité doit être caractérisée par un identifiant unique

A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

Etude des associations

- Une association établit une relation entre deux entités. La relation n'a pas de sens sans les entités qu'elle relie.
- Dans l'exemple on voit que l'association *enseigner* dépend à la fois de la *Matière* et de *Enseignant*

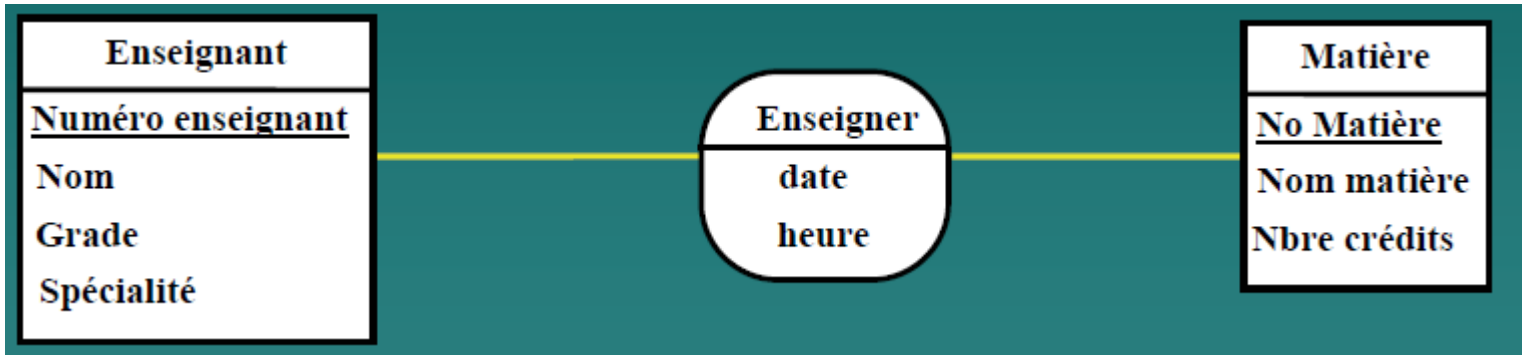


A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

Etude des associations

- On ne représente pas explicitement d'identifiant pour une association. Cependant, implicitement on peut considérer que la concaténation des identifiants des entités reliées joue le rôle d'identifiant pour l'association (voir MCD -> MLD).

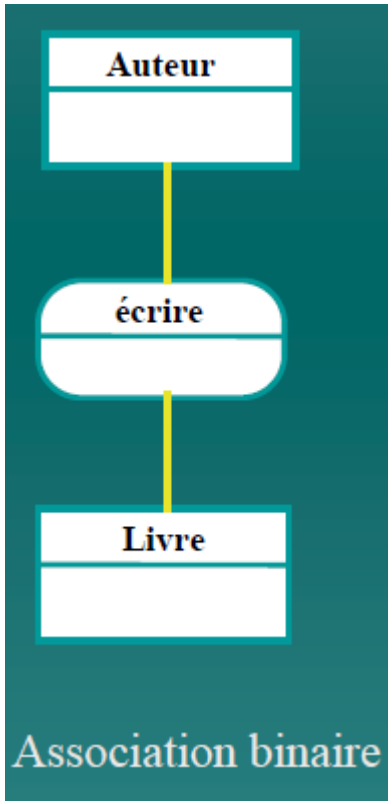


A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

Divers types d'association

Une association binaire concerne 2 entités.



A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

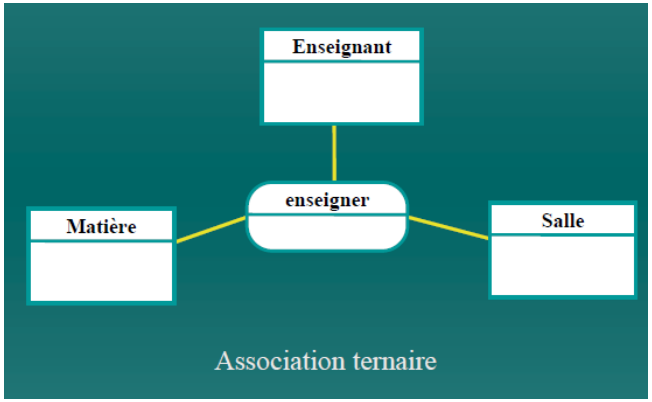
<u>Divers types d'association</u>

Une association peut relier plusieurs entité

On appelle association ternaire

une association qui relie

3 entités.



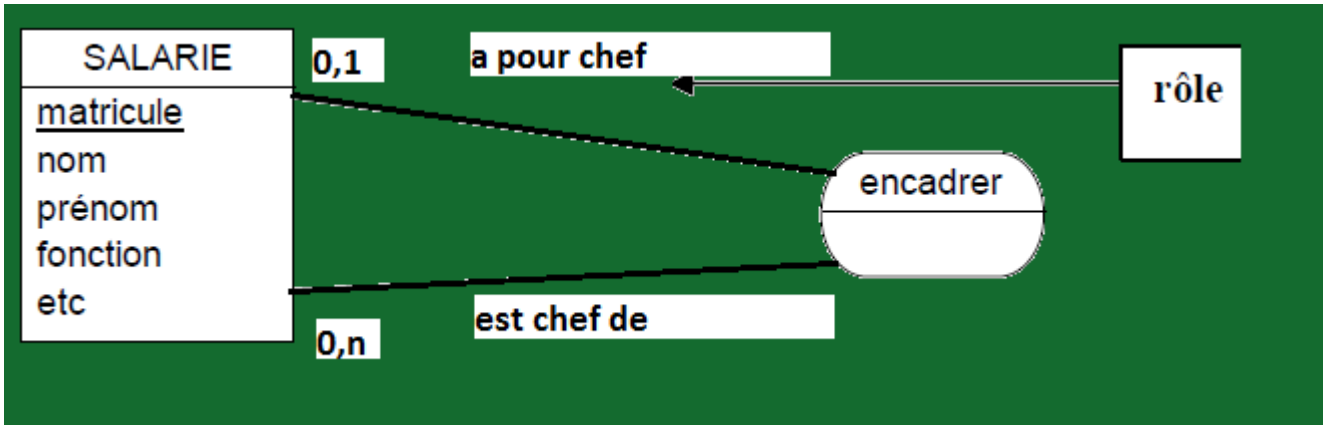
A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

Divers types d'association

Une association réflexive est une association reliant des occurrences de la même entité.

Pour lire une association réflexive, il faut connaitre le rôle de chaque branche de l'association.



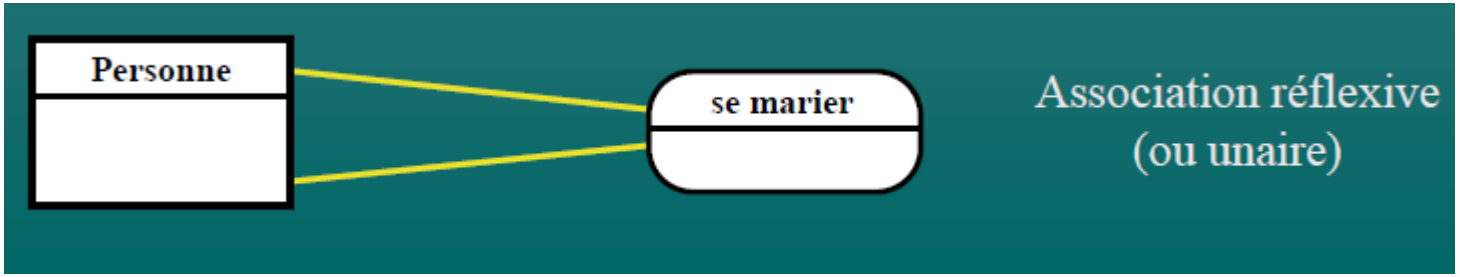
A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

<u>Divers types d'association</u>

Une association réflexive

Quel rôles pour ce exemple?



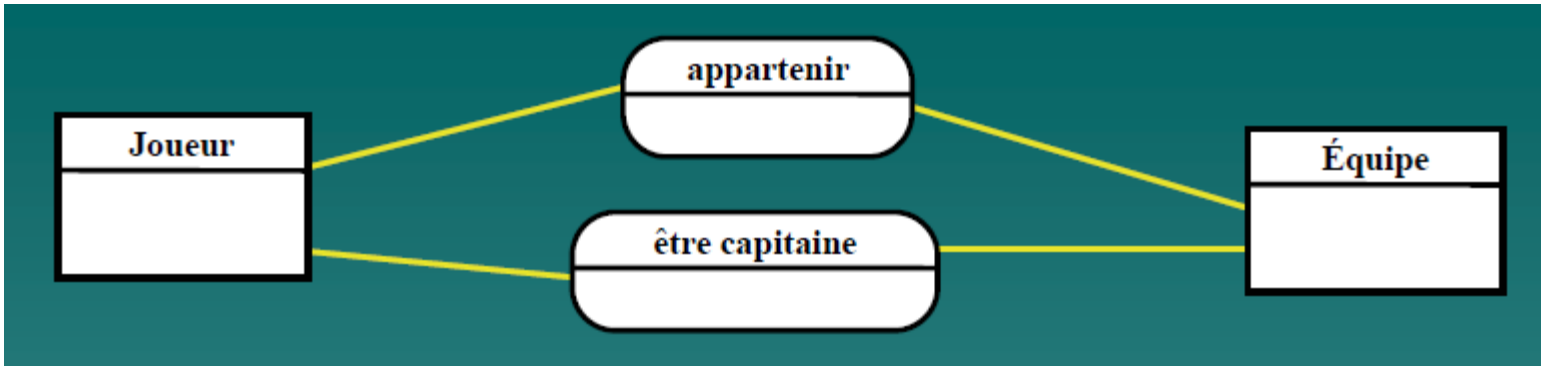
A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

<u>Divers types d'association</u>

Une association plurielle

Deux mêmes entités peuvent être liées par plusieurs associations.



A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

d-Cardinalités d'une association

- Les cardinalités donnent une information sur le nombre d'occurrences possibles de chaque entité impliquée dans une association.
- Exemples :

– Un client passe plusieurs commandes

– Une commande est passée par un seul client

– Un étudiant s'inscrit à plusieurs cours

– Un cours est pris par plusieurs étudiants

A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

d-Cardinalités d'une association

- Identifier le nombre minimal d'occurrences (X) que l'entité peut participer à l'association

  → **Cardinalité minimale**

- Identifier le nombre maximal d'occurrences (Y) que l'entité peut participer à l'association

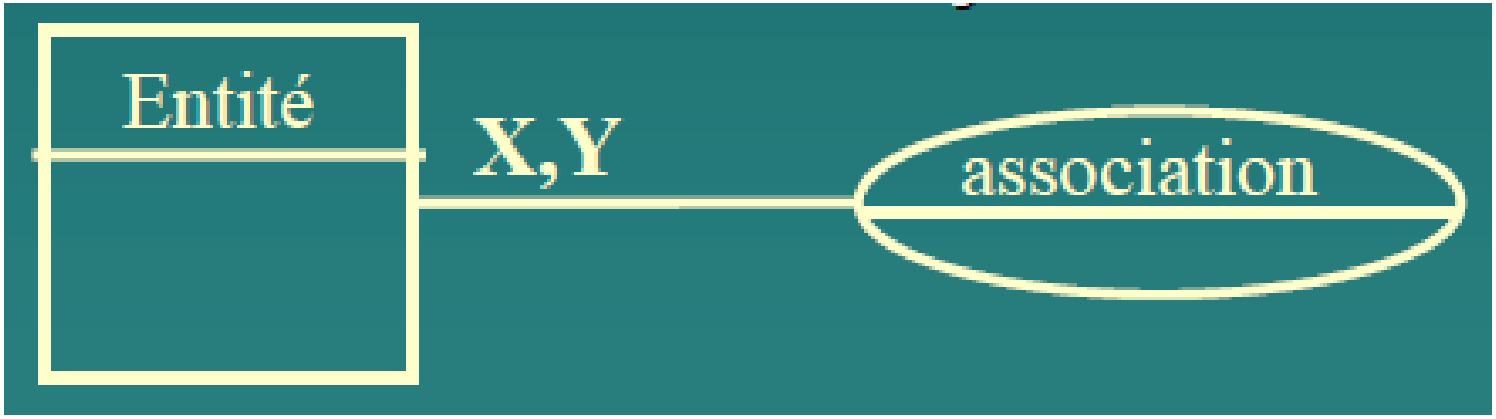
  → **Cardinalité maximale**

A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

d-Cardinalités d'une association

La cardinalité est modélisée de la façon suivante:



A - Le Modèle Conceptuel des Données (MCD)

1 Concepts de base pour le MCD

d-Cardinalités d'une association

- Types de cardinalités possibles :

– 0,1

– 1,1

– 0,n

– 1,n

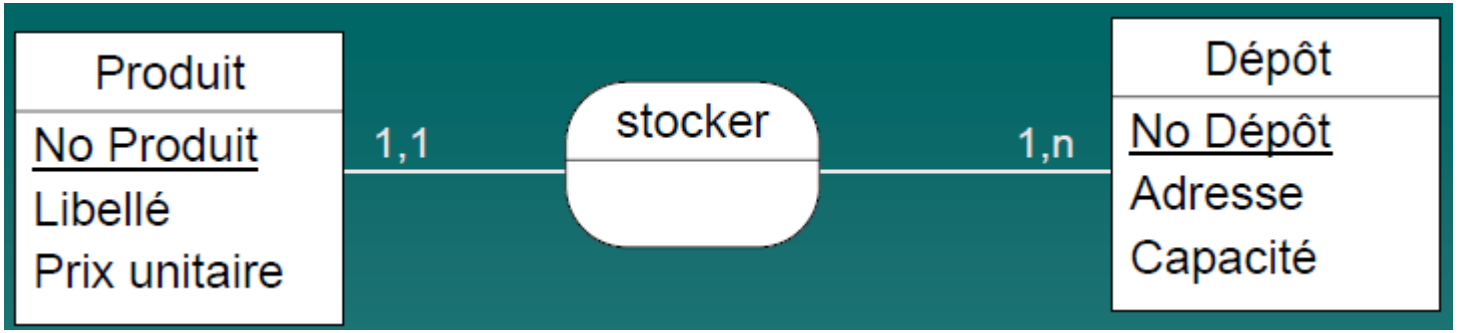
A - Le Modèle Conceptuel des Données (MCD)

d-Cardinalités d'une association

Exemple

– Un produit doit être stocké dans un et un seul dépôt

– Un dépôt doit contenir au moins un produit



- Un *Produit* participe une et une seule fois à l'association *stocker*
- Un *Dépôt* participe une ou plusieurs fois à l'association *stocker*

A - Le Modèle Conceptuel des Données (MCD)

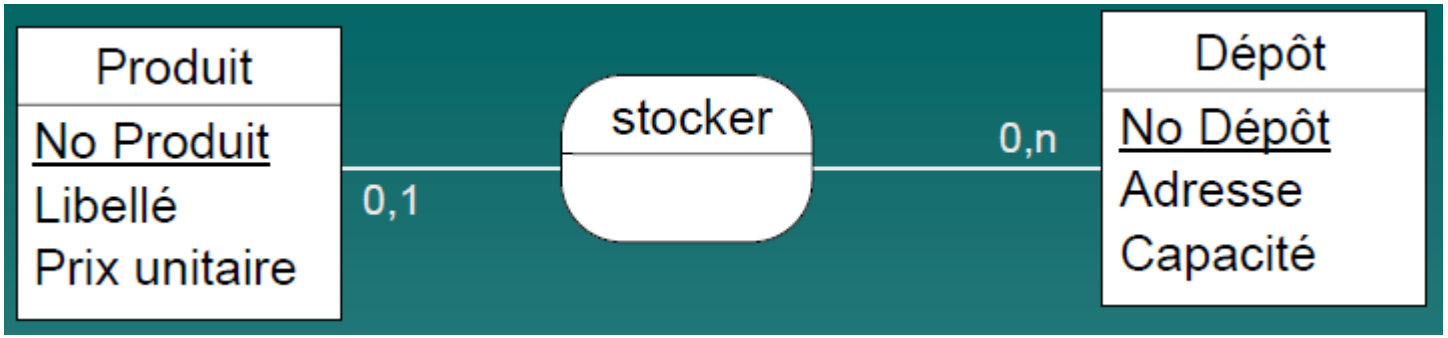
d-Cardinalités d'une association

Exemple

– Un produit peut ne pas être stocké dans un dépôt

– Un produit ne peut être stocké que dans un seul dépôt

– Un dépôt peut ou non contenir des produits

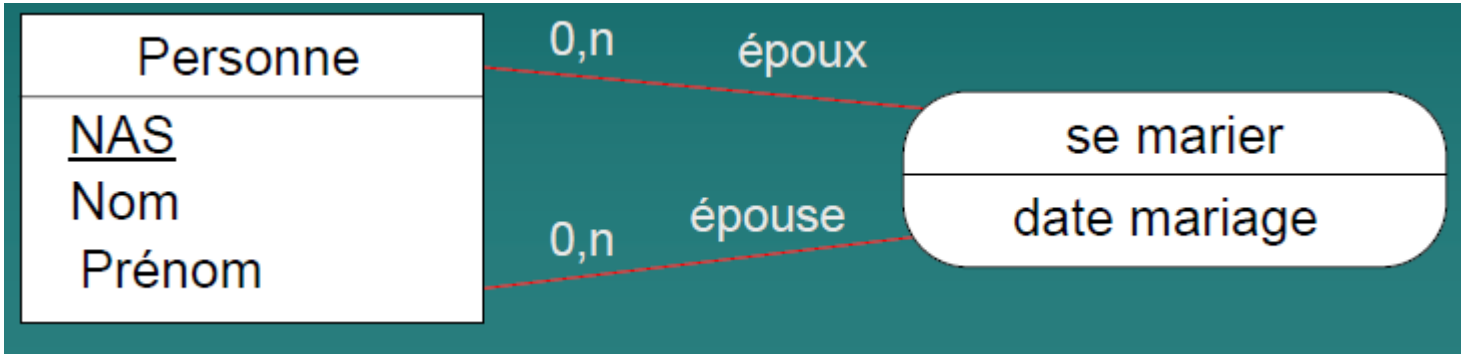


A - Le Modèle Conceptuel des Données (MCD)

d-Cardinalités d'une association

Exemple:

Quelle cardinalité pour la polygamie et la monogamie ?



A - Le Modèle Conceptuel des Données (MCD)

d-Cardinalités d'une association

Exemple:

Un employé travaille pour une entreprise

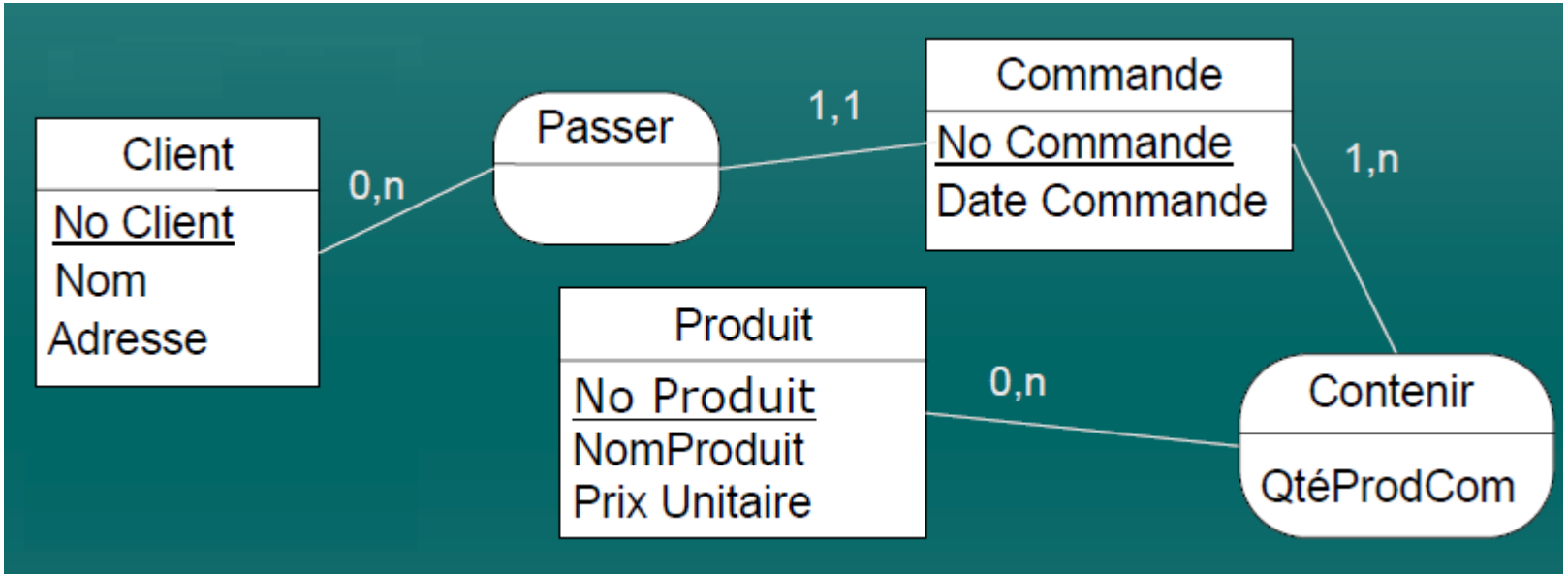
Un employé peut diriger une département

Un département est dirigé par un employé.

A - Le Modèle Conceptuel des Données (MCD)

d-Cardinalités d'une association

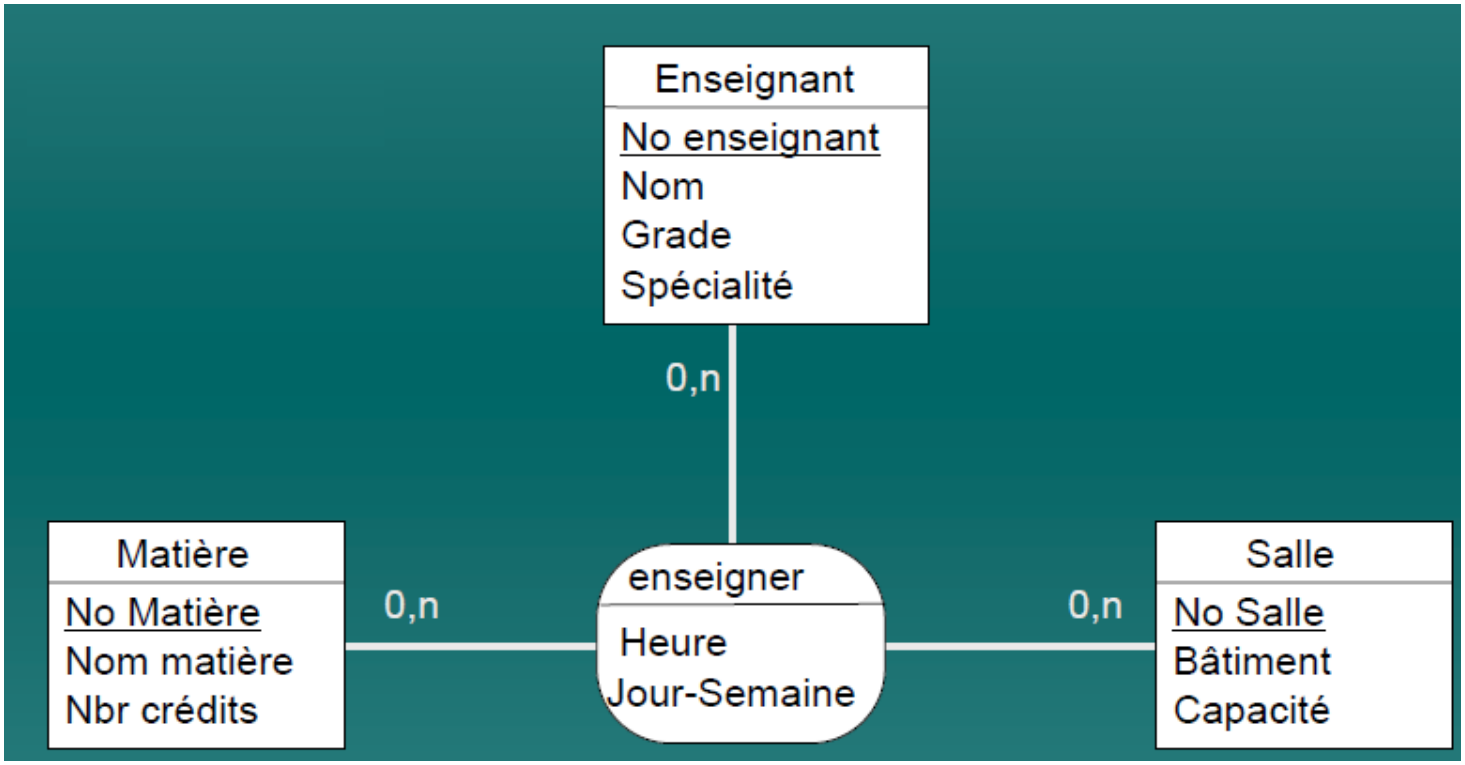
Exemple



A - Le Modèle Conceptuel des Données (MCD)

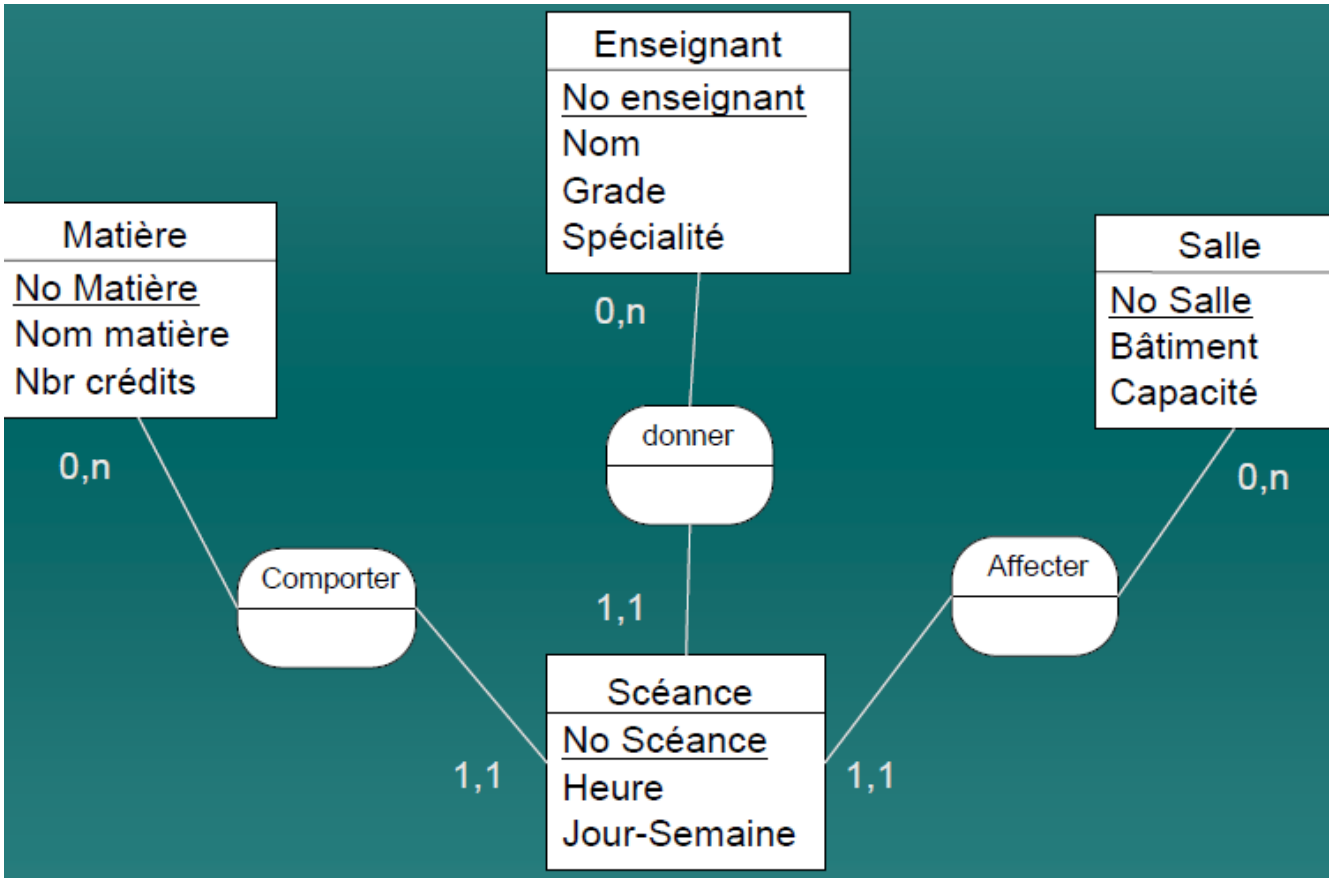
d-Cardinalités d'une association

- Bonne pratique: En général on essaye d'éviter d'utiliser des associations ternaires ou de arité supérieure.



A - Le Modèle Conceptuel des Données (MCD)

d-Cardinalités d'une association



A - Le Modèle Conceptuel des Données (MCD)

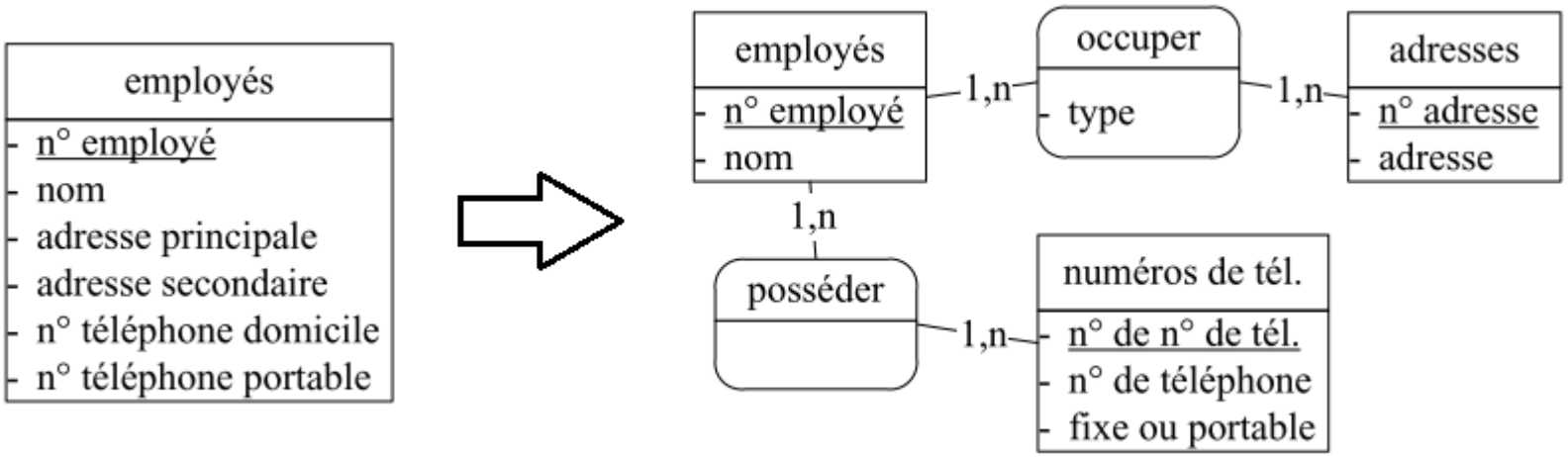
e-Règes de normalisation

- Le nom d’une entité, d’une association ou d’un attribut être unique.
- Un verbe à l’infinitif pour les associations
- Un nom au singulier pour les attributs
- Un nom commun pour les attributs

A - Le Modèle Conceptuel des Données (MCD)

e-Règes de normalisation

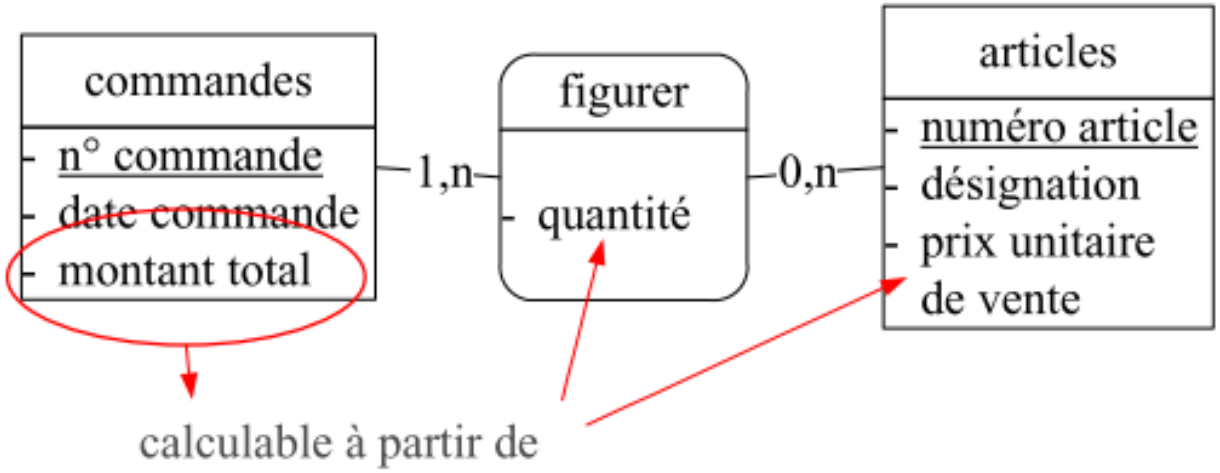
- Remplacer les attributs en plusieurs exemplaires en une association supplémentaire de cardinalités maximales n



A - Le Modèle Conceptuel des Données (MCD)

e-Règes de normalisation

- Ne pas ajouter d'attribut calculable à partir d'autres attributs.



A - Le Modèle Conceptuel des Données (MCD)

<u>Exercice1</u>

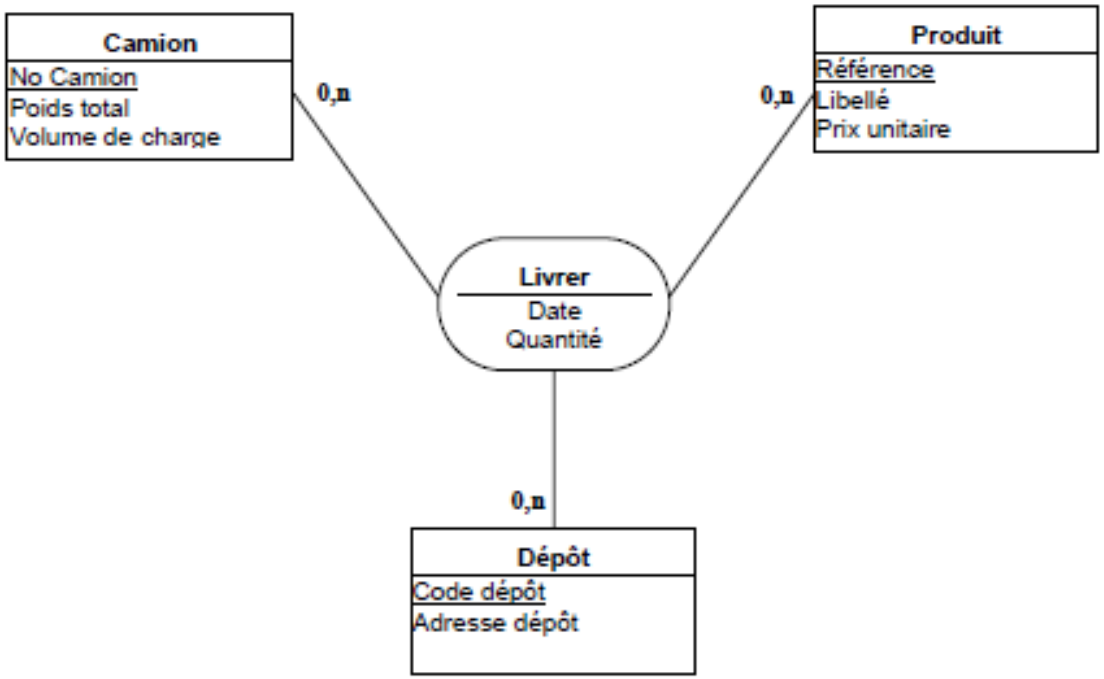
On souhaite modéliser le système d'information des livraisons d'une entreprise. Les informations dont nous disposons sont les suivantes:

Des camions, identifiés par un Numéro de camion et dont on connaît par ailleurs le Poids total en charge et le volume de charge en m^3 livrent des produits dans des dépôts. Les produits ont une référence, un libellé et un prix unitaire, les dépôts possèdent un code de dépôt et une adresse.

Des entretiens menés avec les responsables ont permis de mettre en évidence que les camions, les produits et les dépôts sont indépendants.

Un camion peut livrer n'importe quel produit dans n'importe quel dépôt. Pour chaque action de livraison, on note la date et la quantité.

Question 1 : *Proposer un Modèle Conceptuel des Données.*

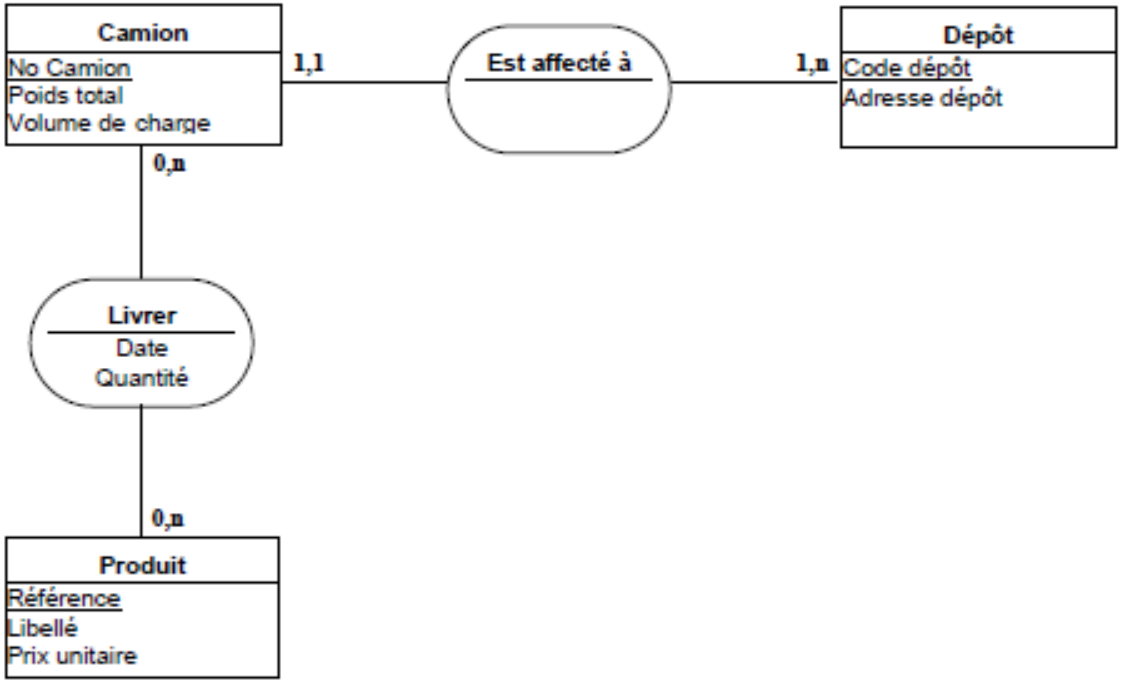

Camion
No Camion
Poids total
Volume de charge
0,n
Produit
Référence
Libellé
Prix unitaire
0,n
Livrer
Date
Quantité
0,n
Dépôt
Code dépôt
Adresse dépôt

A - Le Modèle Conceptuel des Données (MCD)

Exercice1

La pratique a permis de mettre en évidence certains défauts dans l'organisation de l'entreprise (de l’exercice 1). Il apparaît qu'il serait beaucoup plus efficace d'affecter les camions aux dépôts avec lesquels ils travaillent. Ainsi, un camion est affecté à un dépôt donné. Chaque dépôt possède de un à plusieurs camions.

Question 2 : *Modifier le modèle précédent pour prendre en charge cette nouvelle contrainte.*


Camion
No Camion
Poids total
Volume de charge
1,1
Est affecté à
1,n
Dépôt
Code dépôt
Adresse dépôt
0,n
Livrer
Date
Quantité
0,n
Produit
Référence
Libellé
Prix unitaire

A - Le Modèle Conceptuel des Données (MCD)

Exercice 2 (Université)

Le système informatique d'une université utilise les données suivantes :

- Pour chaque étudiant : son numéro de matricule, son nom, son prénom et son adresse (rue, numéro,

cp, ville).

- Pour chaque cours : le mnémonique, l'intitule et un petit résume.
- Pour chaque professeur : son numéro de matricule, son titre, son nom, son prénom, et son adresse.

De plus le système doit savoir quels sont les cours que chaque étudiant suit, le professeur titulaire de

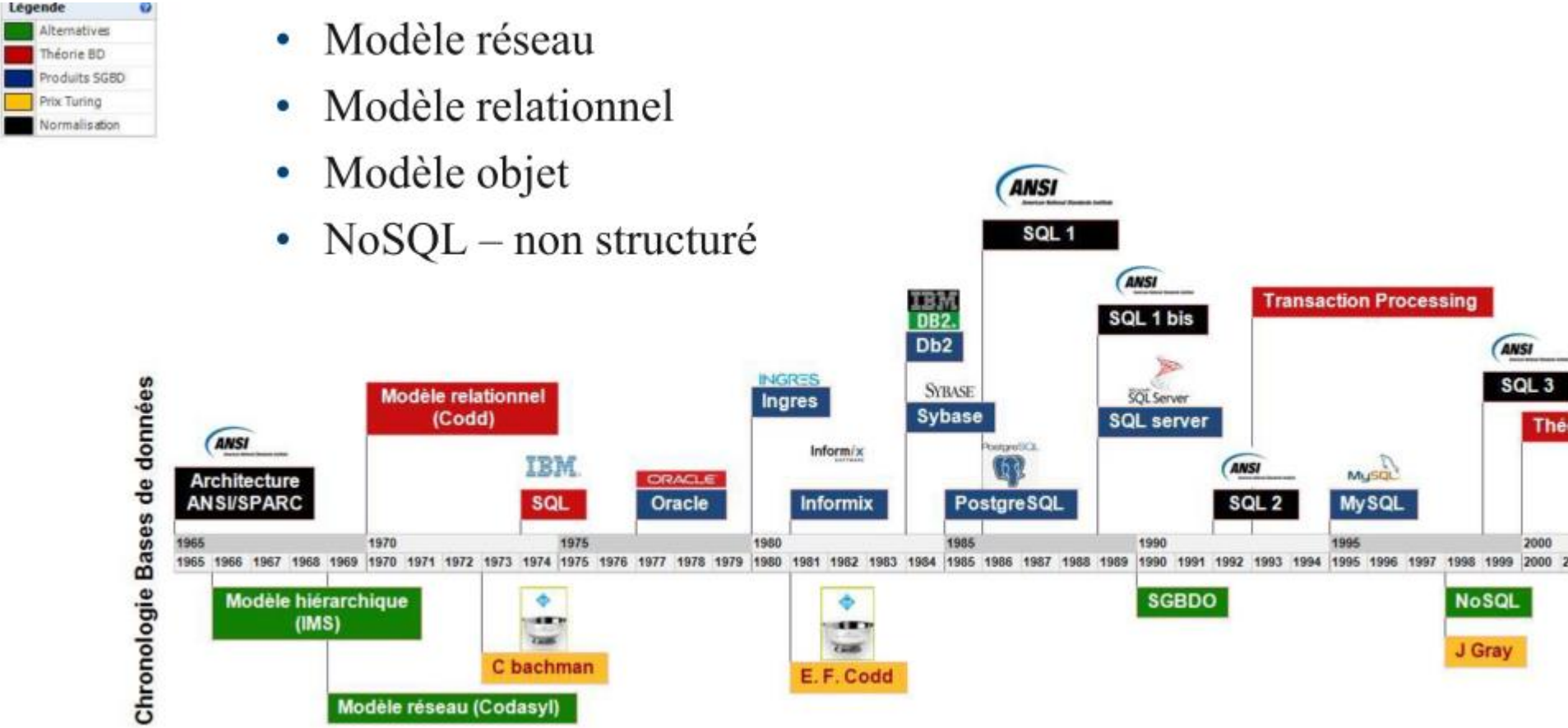
chaque cours, la filière choisie par chaque étudiant et pour chaque filière (dont on retient le nom et le code) le professeur la dirigeant.

Donner un modèle entité-association de ce système et préciser les contraintes d'intégrité.

B - Le Modèle Logique des Données (MLD)

1 Introduction

- Modèle réseau
- Modèle relationnel
- Modèle objet
- NoSQL – non structuré



B - Le Modèle Logique des Données (MLD)

1 Introduction

Il s’agit de traduire le modèle E/A vers un modèle d’implantation à choisir (modèle relationnelle).

B - Le Modèle Logique des Données (MLD)

Principe de traduction

- La sémantique n'est complétement pas préservée.
  - Ajout de contrainte d'intégrité
- Règles automatisables (de nombreux outils existent sur le marché)
  - AMCDesignor, GraphMake, **DBDesigner,…**

B - Le Modèle Logique des Données (MLD)

1 Introduction

Le MCD a permis de modéliser les structures de données indépendamment des choix techniques d'implantation de la base de données (Choix du SGBD).

L'étape suivante : traduire le MCD en un modèle plus proche du SGBD: le modèle logique des données (MLD). Nous considérons ici le cas où le SGBD choisi est de type relationnel.

B - Le Modèle Logique des Données (MLD)

1 Introduction

Dans un MLD basé sur le modèle relationnel, l'unique objet existant est la table.

Le passage du MCD au MLD est systématique

On applique les règles de passage du MCD au MLD.

B - Le Modèle Logique des Données (MLD)

2-Passage du MCD au MLD

- Avec le formalisme Entité-Association, il faut définir tout un ensemble de règles de passage du MCD au MLD. Nous allons examiner les principales dans les diapos suivantes.

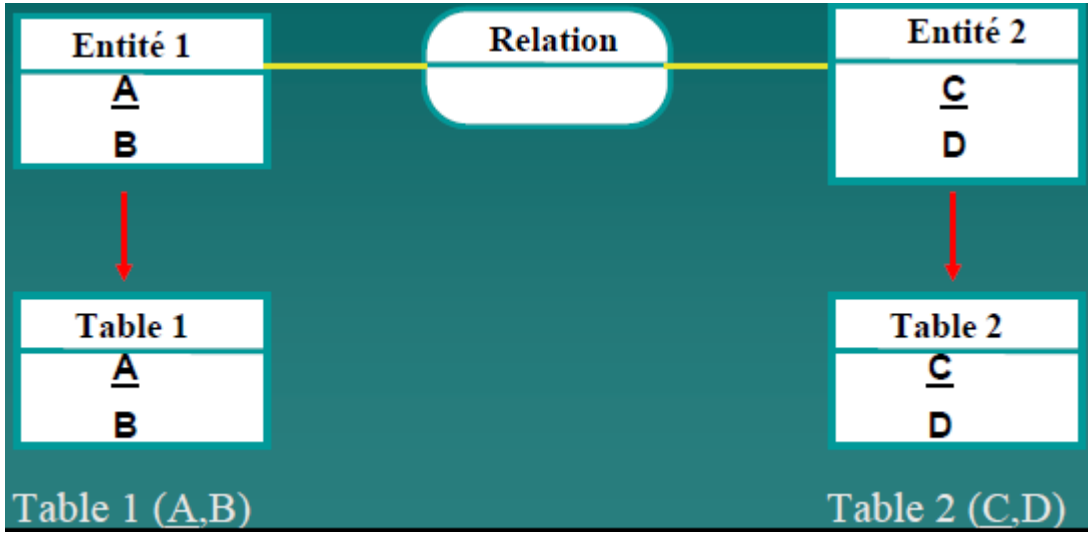
2-Passage du MCD au MLD

Traitement des entités

- Chaque entité devient une table ou relation;
- Chaque propriété (non composite, non calculé) d'une entité devient un attribut de la table correspondante;
- L'identifiant d'une entité devient la clé primaire de la table.

2-Passage du MCD au MLD

Traitement des entités



2-Passage du MCD au MLD

Traitement des associations

- **Cas de l'association père-fils**

Cardinalité de l'entité père : 0,n ou 1,n

Cardinalité de l'entité fils : 0,1 ou 1,1

De façon générale, l'une des cardinalités maximales est 1.

Ce sont des associations binaires monovaluées.

2-Passage du MCD au MLD

Traitement des associations

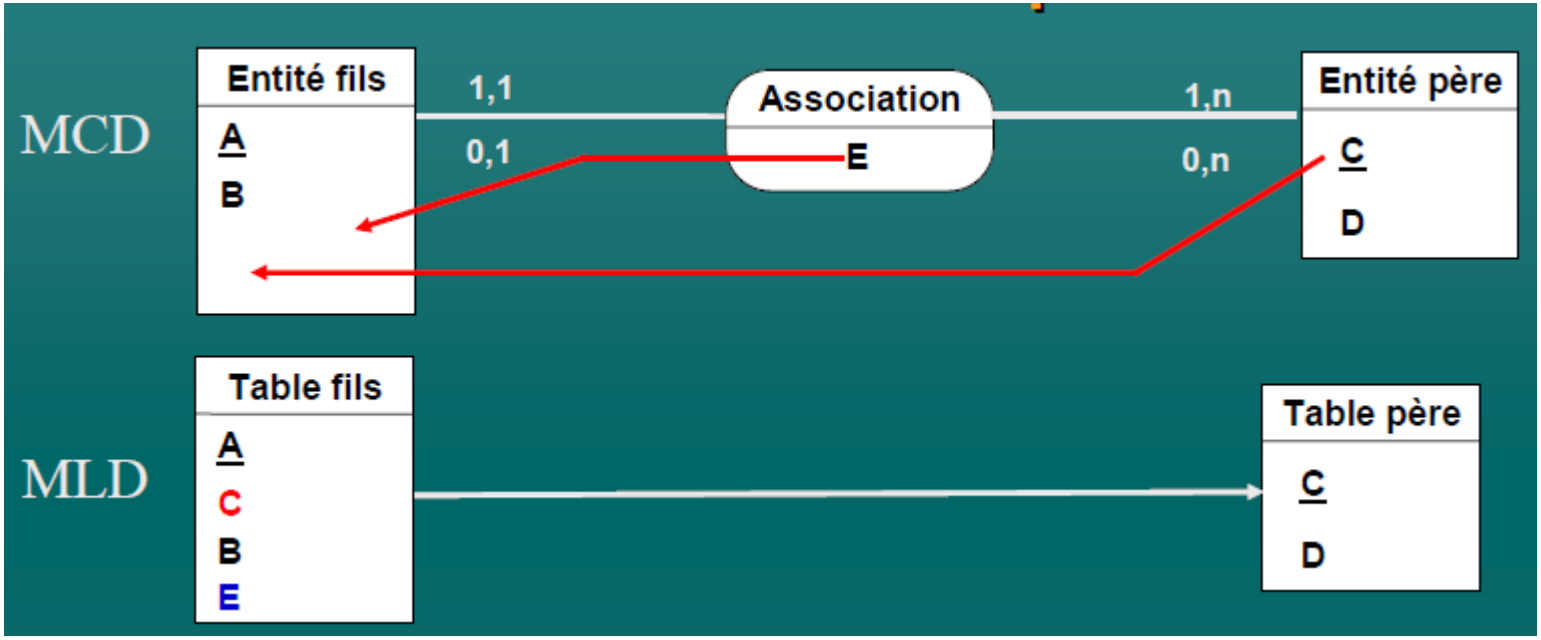
Cas de l'association père-fils (binaire monovaluée)

- L'entité père devient la table « père »
- L'entité fils devient la table « fils »
- L'identifiant de l'entité père devient attribut de la table « fils » (Clé étrangère).
- Les propriétés de l'association deviennent des attributs de la table « fils »

2-Passage du MCD au MLD

Traitement des associations

- **Cas de l'association père-fils(binaire monovaluée)**



2-Passage du MCD au MLD

Traitement des associations

- **Cas de l’association père-fils(binaire monovaluée)**

Cette configuration correspond exactement au cas où on a une dépendance fonctionnelle entre l’entité fils et l’entité père. Si une propriété apparaît dans l’association (ex. E), c’est qu’elle n’était sûrement pas bien placée.

2-Passage du MCD au MLD

Traitement des associations

Exemple:

2-Passage du MCD au MLD

Traitement des associations

- **Cas des associations 0/1 ,n – 0/1,n (binaires multivaluées dans les 2 sens)**

Règles :

–Chaque entité devient une table ayant comme clé son identifiant;

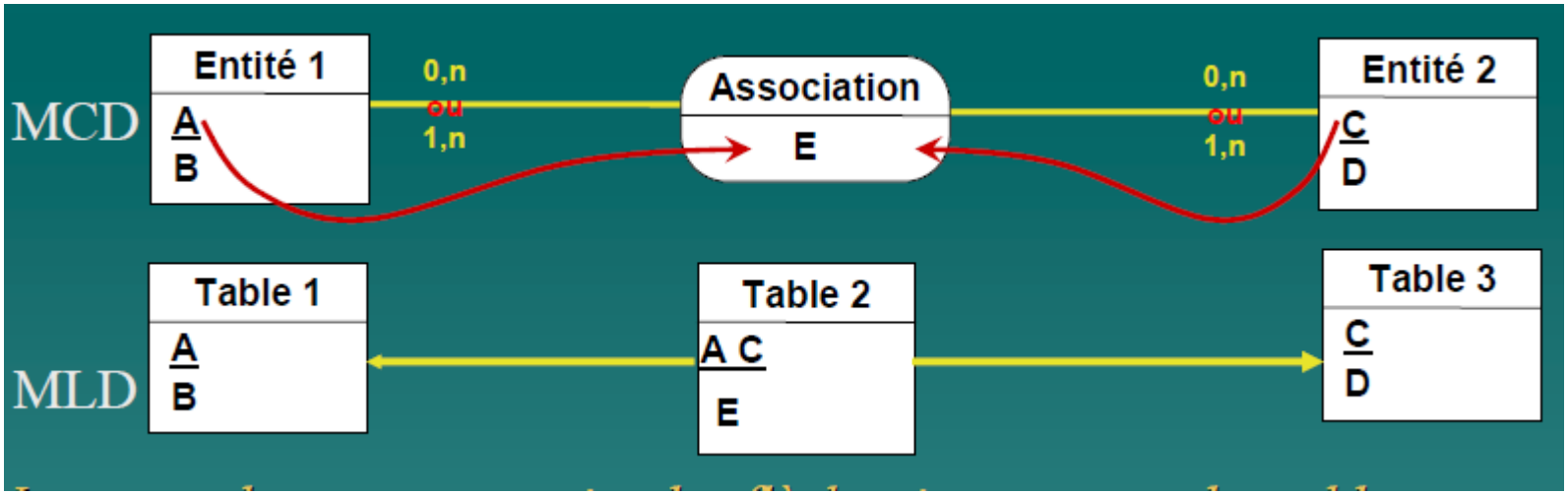
– L’association devient une table;

– L’identifiant de l’association (composé des identifiants des 2 entités) devient la clé de la table correspondante.

2-Passage du MCD au MLD

Traitement des associations

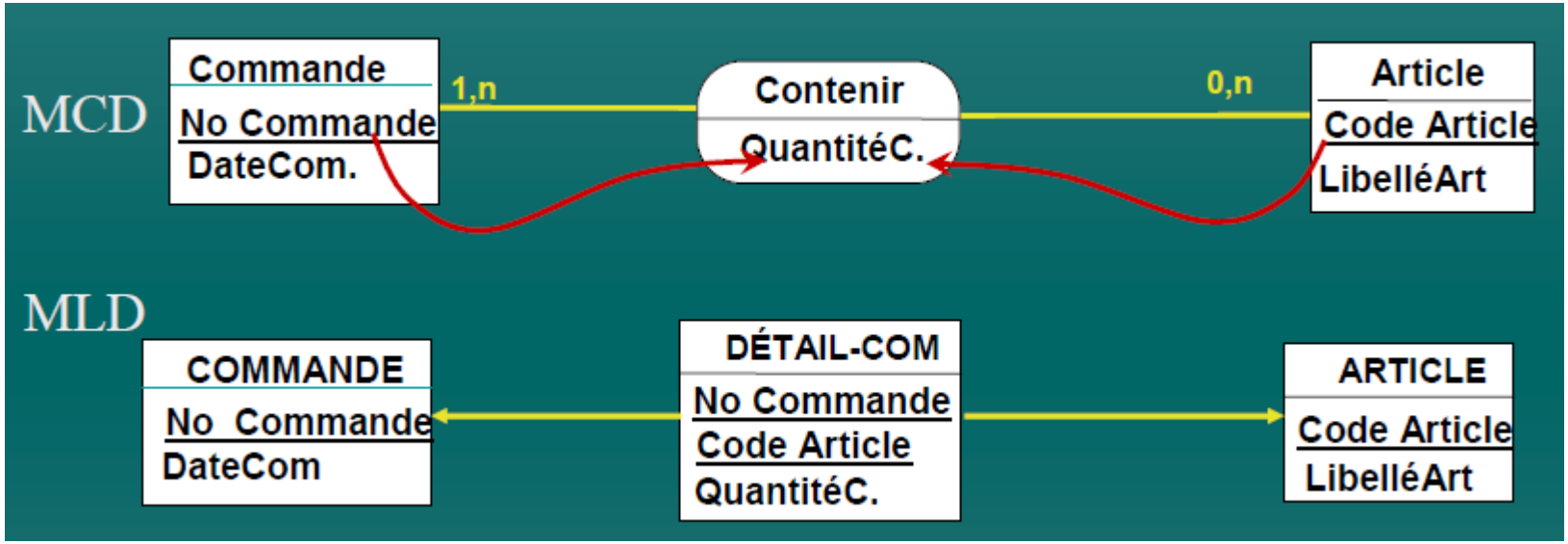
- **Cas des associations 0/1 ,n – 0/1,n (binaires multivaluées dans les 2 sens)**



2-Passage du MCD au MLD

Traitement des associations

- **Cas des associations 0,n – 1,n**



2-Passage du MCD au MLD

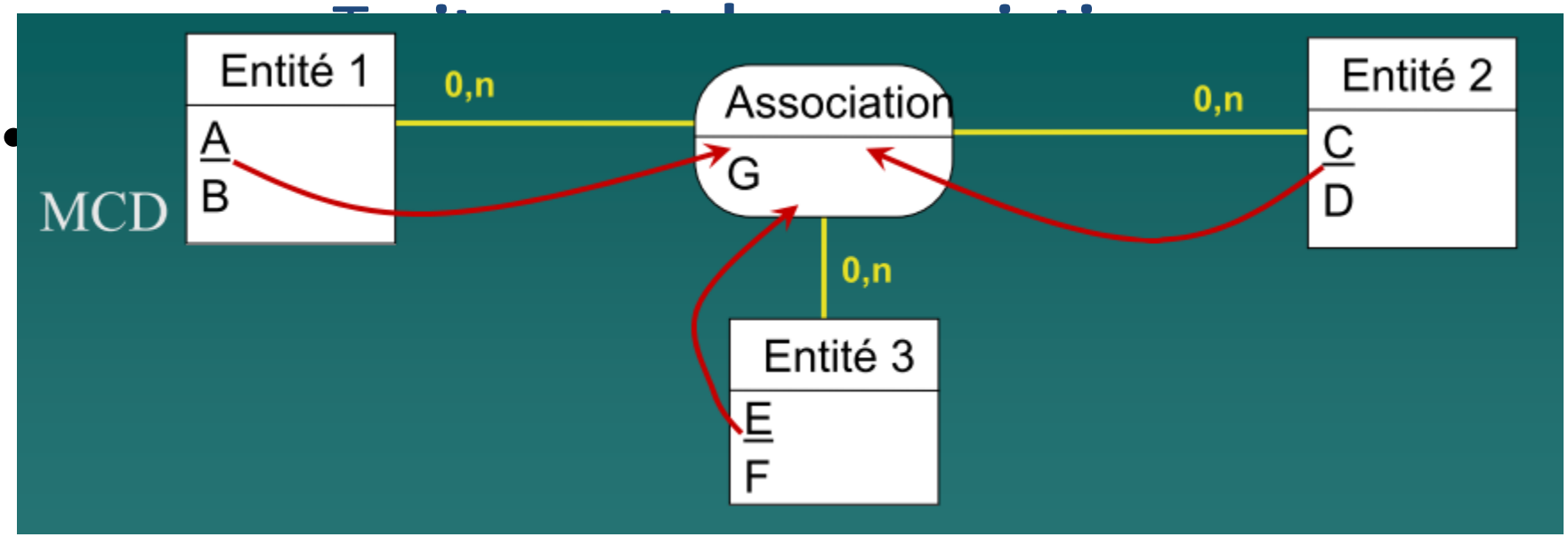
Traitement des associations

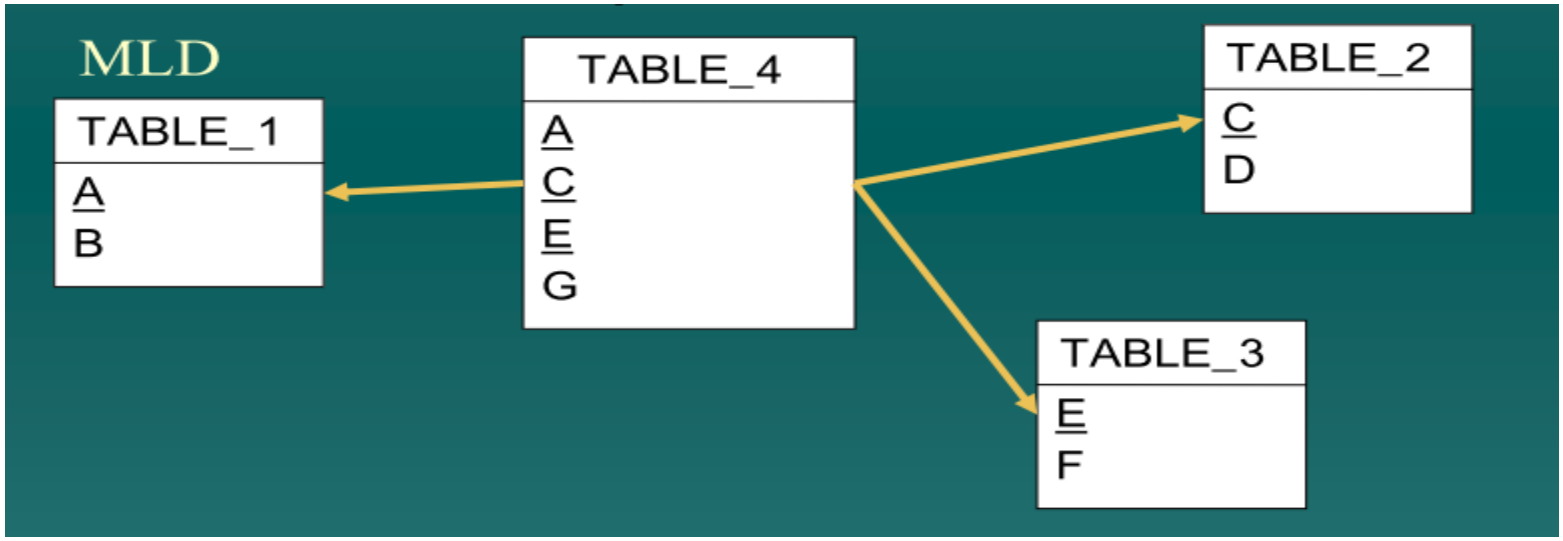
- **Cas association n-aires (n>2) multivaluées dans les 2 sens.**

Le même principe s'applique:

- Création d'une nouvelle table correspondant à l'association
- Les identifiants des entités auxquelles l'association est reliée migrent dans la nouvelle table et forme sa clé primaire
- Les propriétés de l'association sont gardées dans la table correspondante.

2-Passage du MCD au MLD





2-Passage du MCD au MLD

Traitement des associations

- **Cas association n-aires (n>2) multivaluées dans les 2 sens.**

2-Passage du MCD au MLD

Traitement des associations

- **Cas des associations 1,1 – 1,1**

Deux entités reliées par une association de types 1,1- 1,1 devraient être fusionnées en une seule entité, ainsi que les propriétés des relations les reliant.

2-Passage du MCD au MLD

Traitement des associations

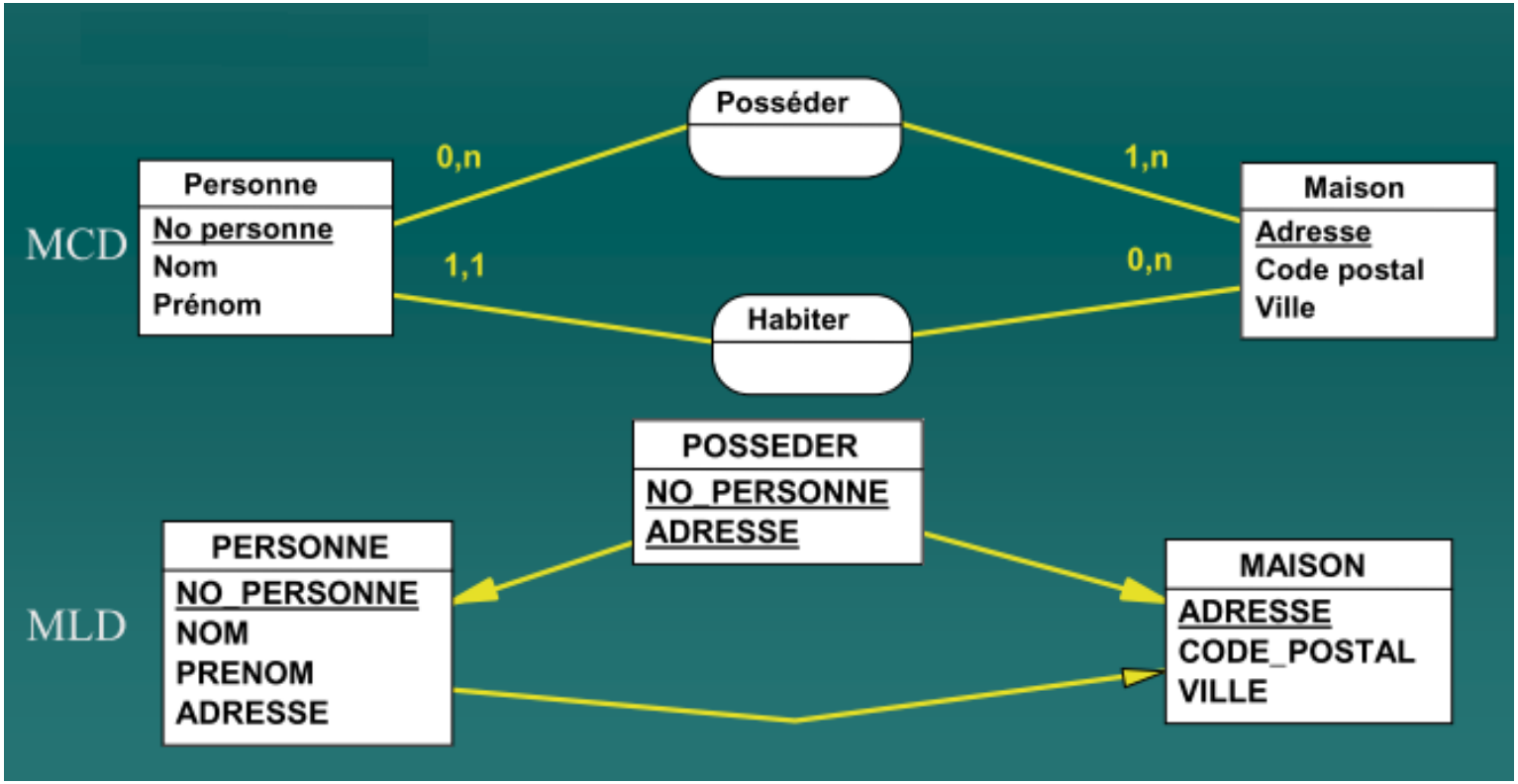
- **Cas des associations 0,1 – 1,1 (association binaire monovaluée)**

Même principe

2-Passage du MCD au MLD

Traitement des associations

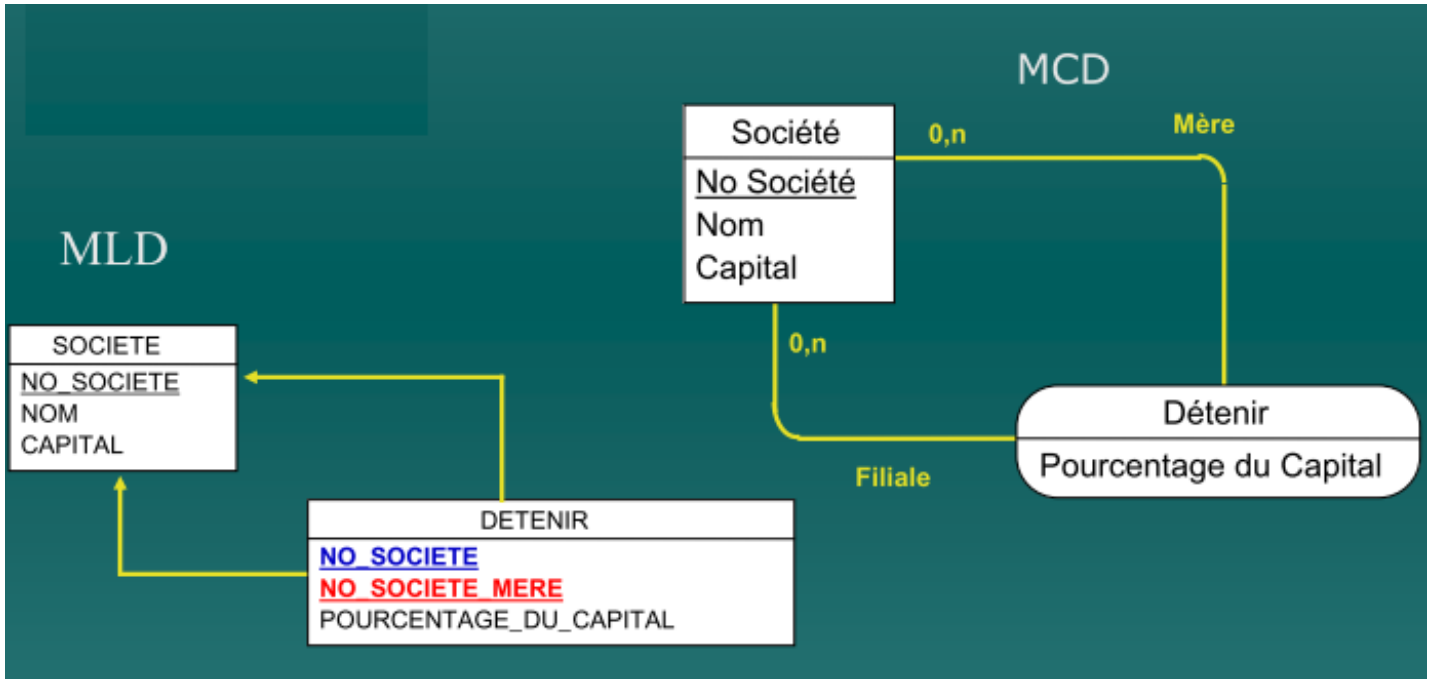
- **Cas spécial de plusieurs associations entre 2 entités.** Les règles générales s'appliquent.



2-Passage du MCD au MLD

Traitement des associations

Cas spécial de relations réflexives : les règles générales s'appliquent.

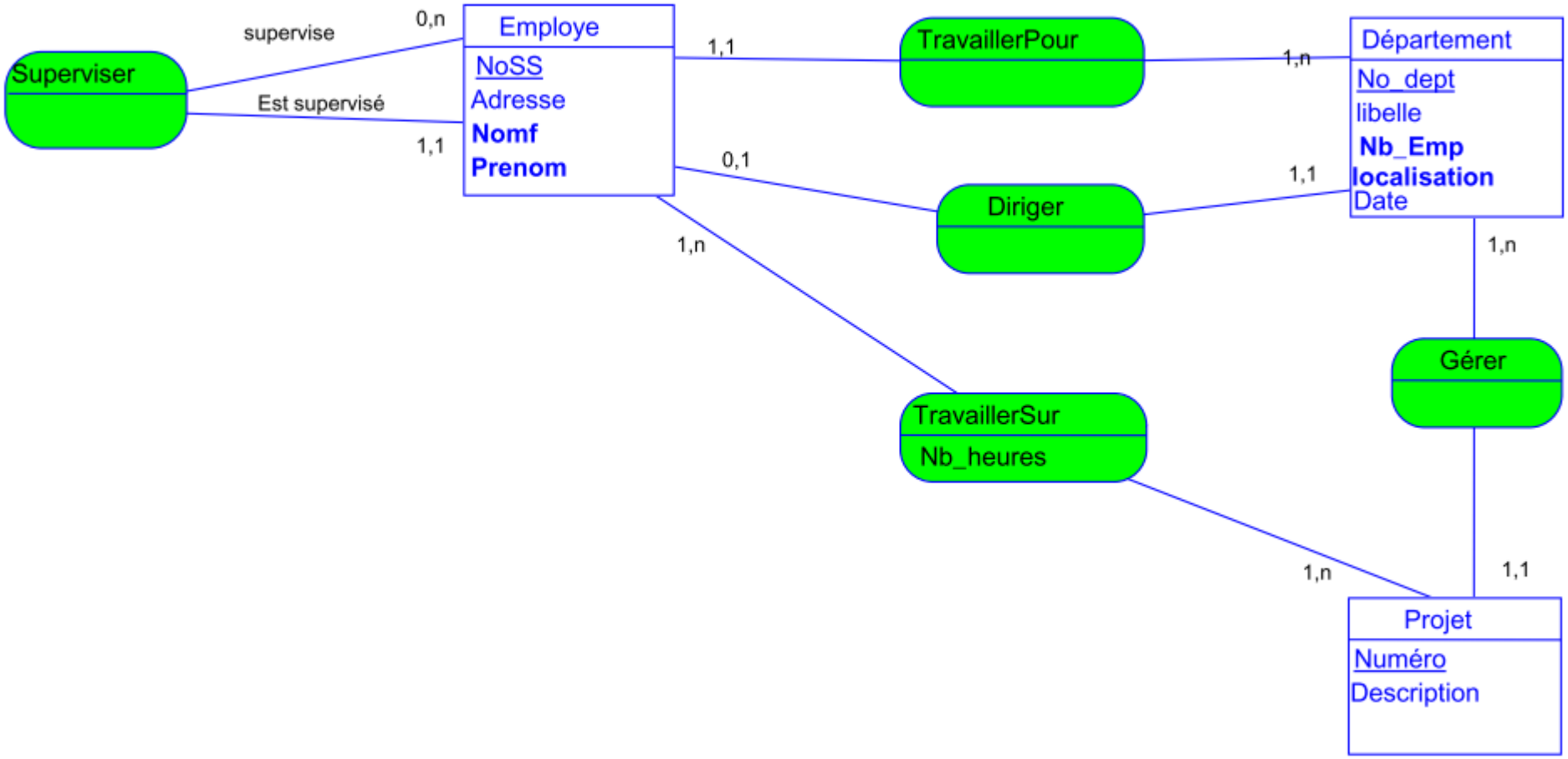


Exercice

Proposer un MLDRelationnel pour chaque MCD des exercices sur le MCD.

Exercice

Traduire en MLD Relationnel ce MCD



Base du SQL

Structured Query Langage

plan

1. Syntaxe générale
2. Requêtes (LMD)
3. MAJ (LMD)
4. Langage de Définition des données (DDL)
5. Introduction à PL/SQL de Oracle
6. Administration

1- Syntaxe générale

L’anglais est le langage du SQL?

SQL implémente les deux grands sous-ensemble dû au modèle relationnel:

- Définition (LDD)
- Manipulation(LMD)
- **Contrôle (DCL)**

1- Syntaxe générale

Le langage de manipulation:

- Déclaratif et non procédurale(description des étapes pas à pas).
- Composition du langage.
  - Algèbre relationnelle+ Fonction-Agrégats+ Tri

1- Syntaxe générale

- **Langage de Manipulation des Données** (LMD)

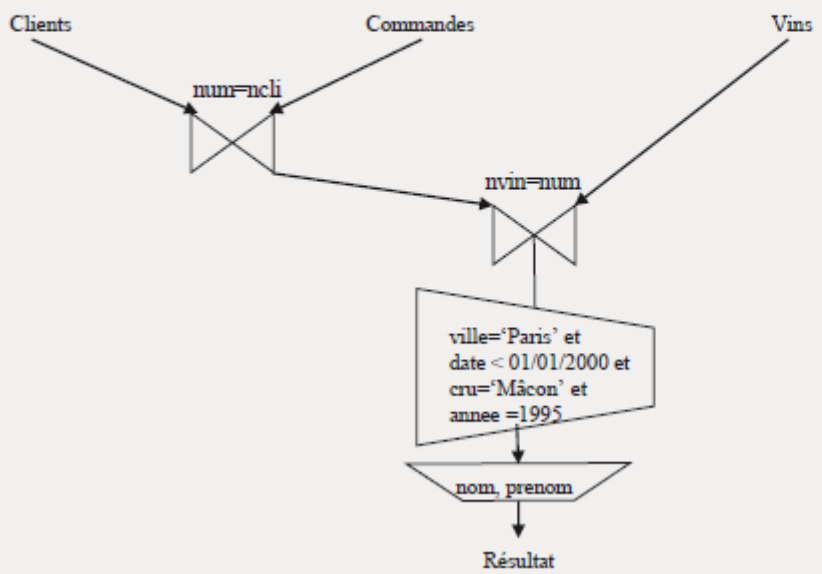
Le langage de manipulation de données (Data Manipulation Language : DML) permet de modifier les données grâce à l'**insertion** (INSERT), la **mise à jour** (UPDATE) , la **suppression** (DELETE) de n-uplets et l'**Interrogation** (SELECT.

- **Langage de définition de données** (LDD)

Le LDD ou (Data Definition Language: DDL) se charge de la définition du schéma d'une base de données.

1- Syntaxe générale

Arbre algébrique



SQL

SELECT

FROM

WHERE

1- Syntaxe générale

Syntaxe

```
SELECT   <liste d'attributs projetés>
FROM     <liste de relations>
[WHERE   <liste des critères
          de restriction
          et de  jointure>]
```

Comment remplir les clauses ?

SELECT : Quel résultat souhaite voir l'utilisateur (schéma du résultat) ?

FROM : Dans quelles relations sont les attributs dont j'ai besoin ?

WHERE : optionnel

- des conditions sur les valeurs d'attributs exprimées dans ma requête ?
- plusieurs relations dans ma clause FROM ? Si oui, quelles sont les conditions de jointure(s) ?

1- Syntaxe générale

Base de données exemple: vins

Vins(num, cru, annee, degre)

Recoltes(nvin, nprod, quantite)

Producteurs(num, nom, prenom, region)

Clients(num, nom, prenom, ville)

Commandes(ncde, date, ncli, nvin, qte)

Livraisons(ncde, no_ordre, qteLivree)

2- Requêtes

Restriction et projection

"Donner les vins de cru Pommard"

Schéma

Vins(num, cru, annee, degre)

```
SELECT

FROM

WHERE
```

2- Requêtes

Restriction et projection

- *"Donner tous les vins"*

```
SELECT    *
FROM      vins
```

- *"Donner la liste de tous les crus, avec élimination des doublons"*

```
SELECT    DISTINCT cru
FROM      vins
```

2- Requêtes

Restriction et projection

Intervalle

- *"Donner les vins de degré compris entre 8 et 12"*

```
SELECT *
FROM Vins
WHERE degre >=8 AND degre <=12 ;
```

```
SELECT *
FROM Vins
WHERE degre BETWEEN 8 AND 12 ;
```

```
SELECT *
FROM Vins
WHERE degre IN (8, 9, 10, 11, 12);
```

2- Requêtes

Restriction et projection

TRI

- ***"Donner les vins dont le cru commence par p ou P"***

SELECT *

FROM Vins

WHERE

- ***"Donner les crus des vins de millésime 1995 et de degré 12, triés par ordre croissant"***

SELECT cru

FROM Vins

WHERE annee=1995 AND degre = 12

2- Requêtes

Restriction et projection

TRI

- ***"Donner les vins dont le cru commence par p ou P"***

SELECT *

FROM Vins

WHERE cru LIKE ‘p%’ or cru LIKE ‘P%’;

- ***"Donner les crus des vins de millésime 1995 et de degré 12, triés par ordre croissant"***

SELECT cru

FROM Vins

WHERE annee=1995 AND degre = 12

ORDER by cry ASC ;

2- Requêtes

Jointure

"Donner les noms des producteurs de Pommard"

Schéma

Vins(num, cru, annee, degre)
Recoltes(nvin, nprod, quantite)
Producteurs(num, nom, prenom, region)

2- Requêtes

Sous-requêtes

2- Requêtes

Sous-requêtes

Une sous-requêtes est une requête à l’intérieur d’une requête. Les sous-requêtes peuvent être dans le WHERE, FROM, ou SELECT.

- Lorsque la sous-requête renvoie une unique valeur, la requête principale peut utiliser cette valeur avec les opérateurs de comparaison classiques

2- Requêtes

Sous-requêtes

```
Select *
From  vins
Where degre >= (Select Max(degre)
                From articles
                where  cru = 'Pommard'
                 );
```

2- Requêtes

Sous-requêtes

Lorsque la sous-requête renvoie une colonne de valeurs, la principale peut utiliser un opérateur de comparaion classique mais accompagné soit de all soit de ANY.

2- Requêtes

Sous-requêtes

Une sous requête peut renvoyer plusieurs colonnes de valeurs, auquel cas elle peut être utilisée(avec un alias de table) dans la clause ***from*** de la requête principale.

2- Requêtes

Sous-requêtes

```
Select *
From
(Select num, cru, dgree
        From vins
        Where annee=2000 ) vins2000
Where cru = ‘mouton cadet’ ;
```

2- Requêtes

Jointure

"Donner les noms des producteurs de Pommard"

Schéma

Vins(num, cru, annee, degre)
Recoltes(nvin, nprod, quantite)
Producteurs(num, nom, prenom, region)

```
SELECT   nom

FROM     vins V,
         producteurs P,
         recoltes R

WHERE    cru = 'Pommard'
         And v.num = nvim
         And P.num = nprod;
```

2- Requêtes

Jointure

Autre formes de jointure(syntaxe SQL2)

- Syntaxe plus proche de l'algèbre relationnelle (directement exprimée dans le FROM). Supportée dans plusieurs SGBD (>= Oracle 9, MySQL, SQLServer, …)

```
SELECT nom
FROM Vins V JOIN Recoltes R ON
(V.num = R.nvin) JOIN Producteurs P
ON (R.nprod = P.num)
WHERE cru = 'Pommard' ;
```

2- Requêtes

Jointure

Autres formes de jointure(procédurale)

```
SELECT nom
FROM Producteurs
WHERE num IN (
   SELECT nprod
   FROM Recoltes
   WHERE nvin IN (
      SELECT num
      FROM Vins
      WHERE cru = 'Pommard')) ;
```

2- Requêtes

Jointure

Auto-jointure : **Jointure d'une relation avec elle-même**

« Donner les couples de producteurs produisant le même vin »

```
SELECT P1.num, P2.num
FROM Producteurs P1, Producteurs P2,
     Recoltes R1, Recoltes R2
WHERE      P1.num = R1.nprod  AND   P2.num =
R2.nprod  AND   R1.nvin = R2.nvin
       AND  P1.num > P2.num ;
```

2- Requêtes

Opérateurs ensemblistes

Union (norme SQL1)

- Élimination automatique des doublons

SELECT num FROM Producteurs

UNION

SELECT num FROM Clients ;

2- Requêtes

Opérateurs ensemblistes

SELECT num FROM Producteurs

INTERSECT

SELECT num FROM Clients ;

Différence

SELECT num FROM Clients

EXCEPT (ou MINUS)

SELECT num FROM Producteurs

2- Requêtes

Fonctions –agrégats

Exemple

Vins	num	cru	degre	annee
	1	Pommard	12	2010
	2	Tavel	11	2009
	3	Pommard		2010
	4	Tavel	12	
	5	Tavel	13	2012
Nb vins				
Nb cru				
Nb annee				
Degre moyen				
Degre min				
Degre max				

2- Requêtes

Fonctions

Fonctions prédéfinies : COUNT, SUM, MIN, MAX, AVG

Principe :

- **S'applique à l'ensemble des valeurs d'un attribut d'une relation**
- **Produit une valeur unique**
- **Pour une requête sans <u>partionnement</u> :**
  - **uniquement dans le SELECT, jamais dans le WHERE**
  - **Ne pas mélanger dans le SELECT les fonctions et les attributs simples!**

num	AVG(degre)
{1, 3, 6, 10}	12,5
{5, 7}	12,0
{2, 8, 11}	11,0

2- Requêtes

Fonctions

Exemple 1 (avg)

« Donner la moyenne des degrés de tous les vins »

```
SELECT avg(degree)
FROM vins;
```

Exemple 2 (sum)

"Donner la quantité totale commandée par le client de nom Some"

```
SELECT SUM(qte)
FROM Commandes, Clients
WHERE Clients.nom= ‘Some'
AND Clients.num=Commandes.ncli ;
```

2- Requêtes

Fonctions

Exemple 3 (count)

Donner le "Nombre de crus différents«

```
SELECT COUNT(DISTINCT cru)
FROM Vins ;
```

Exemple 4

Donner le "Nombre de vins«

```
SELECT COUNT (*)      ou  COUNT (num)
FROM vins
```

2- Requêtes

Fonctions

Exemple

Quel sont les " Vins dont le degré est supérieur à la moyenne des degrés des vins"

```
SELECT *
FROM Vins
WHERE degre > (
        SELECT AVG(degre)
        FROM Vins)
;
```

2- Requêtes

2- Requêtes

Partionnement

Vins	num	cru	degre	annee
	1	Pommard	12	2010
	2	Tavel	11	2009
	3	Pommard	13	2010
	4	Tavel	12	
	5	Tavel	13	2012

Stat cru	cru	Moyenne degre
	Pommard	12,5
	Tavel	11

2- Requêtes

Partionnement

Principe : « je veux des stats par ??? »

- Partitionnement horizontal d'une relation, selon les valeurs d'un attribut ou d'un groupe d'attributs qui est spécifié dans la clause GROUP BY
- Fonctions (sum, max,…)sur les groupes
- Restrictions sur les groupes
  - Application possible d'un critère de restriction sur les groupes

  obtenus
  - Clause HAVING

2- Requêtes

Partionnement

" Donner, pour chaque cru, la moyenne des degrés des vins de ce cru …"

```
SELECT CRU, avg(degre)
FROM Vins
GROUP BY CRU
```

… avec un tri par degré décroissant"

```
SELECT cru, AVG(degre)
FROM Vins
GROUP BY cru
ORDER BY AVG(degree) DESC;
```

2- Requêtes

Partionnement

" Donner, pour chaque cru, la moyenne des degrés des vins de ce cru, uniquement si ce cru concerne plus de 3 vins"

```
SELECT cru, AVG(degre)
FROM Vins
GROUP BY cru
HAVING count(*) >= 3
```

Dans le HAVING, les conditions sont exprimés sur des fonctions et pas sur des attributs.

2- Requêtes

Partionnement

Etape de Calcul du partionnement

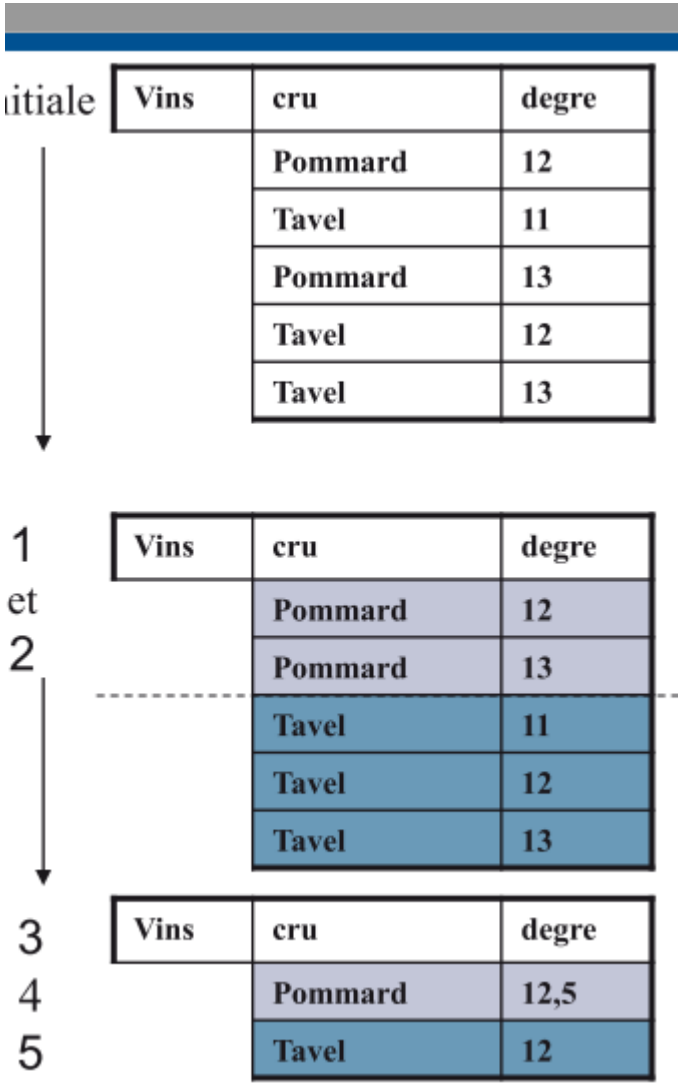
1 Trier la relation **selon les attributs de groupement**

2. Créer **une sous-relation pour chaque paquet** ayant même valeur sur l'ensemble des attributs de groupement, ici « cru »

3. Calculer les fonctions (clause SELECT ou HAVING ou ORDER BY) sur chaque partition (dans notre exemple la valeur de cru et la moyenne des degrés sur la partition)

4. Appliquer la restriction du HAVING

5. Unifier les résultats



2- Requêtes

Partionnement

Exemple de requête erronée

```
SELECT cru, num, AVG(degre)
FROM Vins
GROUP BY cru ;
```

Résultats

cru	num	AVG(degre)
Pommard	{1, 3, 6, 10}	12,5
Tavel	{5, 7}	12,0
Gamay	{2, 8, 11}	11,0

2- Requêtes

Partionnement

Syntaxe

```
SELECT exp1, exp2, ... expn,
aggregate_function(aggregate_expression)
FROM tables
WHERE conditions
GROUP BY exp1, exp2, ... expn;
HAVING   having_condition;
```

2- Requêtes

Prédicat

ALL

Teste si la valeur d'un attribut satisfait un critère de comparaison avec **tous** les résultats d'une sous-requête

Exemple:

Quels sont les clients qui ont passé la plus grosse commande

```
SELECT Cl.num, Cl.nom
FROM Clients Cl, Commandes C
WHERE Cl.num = C.ncli
AND C.qte >= ALL(
      SELECT qte
      FROM Commandes) ;
```

Clients(num, nom, prenom, ville)
Commandes(ncde, date, ncli, nvin, qte)

2- Requêtes

Prédicat

ANY

Teste si la valeur d'un attribut satisfait un critère de comparaison avec **au moins un** résultat d'une sous-requête

Exemple:

Quels sont les clients qui ont passé la plus grosse commande

```
SELECT Cl.num, Cl.nom
FROM Clients Cl, Commandes C
WHERE Cl.num = C.ncli
AND C.qte > ANY(
      SELECT qte
      FROM Commandes) ;
```

Clients(num, nom, prenom, ville)
Commandes(ncde, date, ncli, nvin, qte)

3 - Mise à jour (LMD)

Le LMD permet d’ajouter/modifier/supprimer (mise à jour) les données dans la base de données.

On peut remplir automatiquement la base de données, sous la forme d'instructions écrites dans un fichier.

3 - Mise à jour (LMD)

Insertion de données

Une insertion de n-uplets a la syntaxe suivante :

```
INSERT INTO <table> [(<liste d'attributs>)] VALUES (<liste de valeurs>) | <requete>
```

On peut préciser la liste des attributs à renseigner : les autres valeurs sont alors fixées soit à la valeur par défaut de l'attribut, soit à NULL.

3 - Mise à jour (LMD)

Insertion de données

Exemple.

- `INSERT INTO VINS VALUES (100, 'JURANÇON',1987,12);`

- `INSERT INTO VINS(NUM, CRU, ANNEE, DEGREE) VALUES (100, 'JURANÇON',1987,12);`

- `INSERT INTO VINS(NUM, CRU) VALUES (200,'GAMMAY');`

3 - Mise à jour (LMD)

Insertion de données

La clause VALUES ajoute un n-uplet à la fois alors que l'utilisation d'une requête à la place de VALUES permet d'insérer plusieurs n-uplets en une seule opération.

```
INSERT INTO <table> [(<liste d'attributs>)] <requete>
```

3 - Mise à jour (LMD)

Insertion de données

CREATE TABLE BORDEAUX(num NUMBER, annee DATE, degre NUMBER(4,2)) ;

INSERT INTO BORDEAUX

SELECT num, annee, degre

FROM Vins

WHERE cru = 'Bordeaux' ;

3 - Mise à jour (LMD)

Modification

L'instruction UPDATE modifie les valeurs des n-uplets d'une table :

```
UPDATE <table> SET <liste d'affectations> [WHERE <condition>].
```

L'utilisation d'une liste de valeurs permet de modifier la valeur de plusieurs attributs. La clause WHERE définit les n-uplets qui seront mis à jour.

3 - Mise à jour (LMD)

Mise à jour

Exemple

« Le producteur 150 habite maintenant dans le sahel »

UPDATE Producteurs

SET region = ‘Sahel'

WHERE num = 150 ;

Producteurs(**num**, nom, prenom, region)

3 - Mise à jour (LMD)

Mise à jour

Exemple

« Les degrés des Gamays augmentent de 10 % »

```
UPDATE Vins
SET degre = degre * 1.1
WHERE cru = 'Gamay' ;
```

3 - Mise à jour (LMD)

Mise à jour

Exercice

Les clients de nom « Diallo » augmente les quantités de leurs commandes de 10 unités. *Ecrire la commande qui permet de faire cette Maj.*

3 - Mise à jour (LMD)

Suppression

L'instruction DELETE supprime des n-uplets d'une table :

DELETE FROM <table> [WHERE <condition>].

La clause WHERE précise les n-uplets à supprimer. Sans elle, tout le contenu de la table est supprimé.

3 - Mise à jour (LMD)

Suppression

Exemple

- Supprimer tous les tuples de Vins

```
DELETE FROM Vins ;
```

- Supprimer le vin de numéro 150

```
DELETE FROM Vins WHERE num = 150
;
```

3 - Mise à jour (LMD)

Suppression

Exemple:

- "Supprimer les vins de degré inférieur à 9 ou supérieur à 12 »
- "Supprimer les commandes passées par Mohamed et Hamidou"

4 - Définition de données (LDD)

Le langage de définition de données est la partie de SQL qui permet de décrire les tables et autres objets du SGBD Oracle.

Il propose trois opérations :

- la création (CREATE),
- la suppression (DROP) et
- la modification (ALTER).

4 - Définition de données (LDD)

Création de table

La création d'une table a pour syntaxe :

```
CREATE TABLE nom_table (
colonne1 type [contrainte],
colonne2 type [contrainte],
…
colonneN type [contrainte]
);
```

Il permet de créer et de définir une table

4 - Définition de données (LDD)

Création de table

Chaque définition de colonne comporte le nom de la colonne, son type et éventuellement une spécification de contraintes d'intégrité (not null).

4 - Définition de données (LDD)

Création de table

Les types de données sont :

<u>Chaînes de caractères</u>

- **Char** : un seul caractère;
- **Char(n)** : chaîne de longueur fixe n ($1 \leq n \leq 2000$)
- **Varchar2(n)** : chaîne de longueur variable n ie ajustée au contenu ($1 \leq n \leq 4000$).

4 - Définition de données (LDD)

Création de table

Nombre

- **number(p)** : entier à p chiffres ($1 \leq p \leq 38$)
- **number(p,s)** : nombre réel à virgule fixe de p chiffres dont s chiffres décalés après la virgule ($1 \leq p \leq 38$, $-84 \leq s \leq 127$):
- **number** : nombre réel à virgule flottante avec environ 16 chiffres significatifs.

4 - Définition de données (LDD)

Création de table

Date et heure

Date : entier décrivant le nombre de secondes écoulées depuis le 01/01/-4712 0:00:00.

Et va jusuq'au 31/12/9999.

Sysdate: fonction qui retourne la date et l'heure courante du serveur oracle

4 - Définition de données (LDD)

Création de table

Format de la date:

to_date : fonction permet de convertir date en format (yyyymmdd h:m:s)

to_date : fonction permet de convertir date en format (yyyymmdd h:m:s) chaîne compréhensible par l’homme.

TO_DATE('2003/07/09', 'yyyy/mm/dd')

4 - Définition de données (LDD)

Création de table

Exemple

```
CREATE TABLE etudiant (
 Etud_id  number(10) NOT NULL,
Etud_name varchar2(50) NOT NULL,
city varchar2(50)
 );
```

4 - Définition de données (LDD)

Modification d'une table

Une table qui existe déjà peut être modifiée par l'intruction **ALTER TABLE** accompagné d'une clause **ADD** ou **MODIFY**

Syntaxe:

ALTER TABLE nomtable **MODIF**(*col1 type1, col2 typ2,…*)

Col1, col2,… sont les noms des colonnes que l'on veut modifier. Elle doivent déjà exister dans la table.

Type1, typ2 sont les nouveau types que l'on veut attribuer au colonnes.

4 - Définition de données (LDD)

Modification d'une table

Syntaxe:

ALTER TABLE `nomtable` **ADD** `(col1 type1, col2 typ2, …)`

Cette instruction permet d'ajouter une ou plusieurs colonnes à une table existante.

L'attribut 'not null' n'est possible que si la table est vide sinon la nouvelle colonne sera nulle dans ces lignes existante

4 - Définition de données (LDD)

Modification d’une table

On peut supprimer une colonne d’une table.

Synthaxe:

```
ALTER TABLE nomtable DROP COLUMN
nomcolonne;
```

4 - Définition de données (LDD)

Suppression de table

On peut supprimer une table avec l'instruction

```
DROP TABLE nomtable ;
```

C'est toute table (schéma et données) qui est détruite. L'espace occupé par la table est libéré.

4 - Définition de données (LDD)

Suppression de table

- `Select table_name from user_tables`

Cette instruction permet à l'utilisateur d'Oracle de retrouver toutes ses tables.

- `Desc[ribe] nomtable`

Voir les colonnes d'une table.

4 - Définition de données (LDD)

Vues ou View

On peut enregistrer une requête SELECT en tant que vue. Les utilisateurs peuvent consulter ou modifier la base à travers les vues: C'est-à-dire manipuler la table résultat du select comme si c'était une table réelle.

Une vue est une table virtuelle qui n'existe pas physiquement car seule le schéma est enregistré et pas les données.

4 - Définition de données (LDD)

Vues ou View

Syntaxe de création d'une vue:

CREATE VIEW nomVue AS

```
        SELECT colonne
        from tables
        where conditions;
```

DROP VIEW nomVue**;**

Supprime la vue nomVue

4 - Définition de données (LDD)

Types de Clé

- La **clé primaire** d'une table est un ensemble d'attributs (souvent un seul attribut). Une combinaison de valeurs de ces attributs détermine au plus un n-uplet de la table.
- Une table peut avoir plusieurs clés. On dira d'une clé d'une table T qu'elle est une **clé primaire** si elle est utilisée dans d'autres tables pour identifier des n-uplets de T. (Les autres clés seront dites **clés candidates**.)

Le langage de définition de données (LDD)

Types de clé

- On appelle **clé étrangère** un ensemble d'attributs d'une table T_b faisant référence à la **clé primaire** de la table T_a.

4 - Définition de données (LDD)

Contrainte d’intégrité

- Le langage DDL permet de spécifier qu'un ensemble d'attributs est clé primaire (PRIMARY KEY), qu'une séquence d'attributs est clé étrangère (FOREIGN KEY/REFERENCES).
- Il permet aussi de contraindre le domaine d'un attribut (CHECK) ou de vérifier qu'un ensemble d'attributs ne contient pas de valeur manquante (NOT NULL).
- il permet, de préciser qu'un ensemble d'attributs ne contienne pas de doublons (UNIQUE) ; à la différence de PRIMARY KEY, il est possible de spécifier plusieurs UNIQUE sur la même table.

4 - Définition de données (LDD)

Contrainte d'intégrité (clé)

Exemple:

```
CREATE TABLE coordonnees(
  salle VARCHAR(30) PRIMARY KEY,
  adresse VARCHAR(255) UNIQUE NOT
NULL,
  telephone VARCHAR(10)
);
```

L'attribut salle est la **clé primaire** donc annoté par PRIMARY KEY. Les contraintes UNIQUE et NOT NULL sur adresse précisent que cet attribut est une **clé candidate**.

4 - Définition de données (LDD)

Primary keys

Une clé primaire peut etre définie lors de la création de la table (CREATE) ou en modification (ALTER TABLE)

Une clé primaire peut contenir 32 colonnes (Oracle)

4 - Définition de données (LDD)

Primary keys

Syntaxe:

CREATE TABLE table_name (

column1 datatype null/not null,

column2 datatype null/not null,

...

CONSTRAINT constraint_name PRIMARY KEY (column1, column2, ... column_n)

);

4 - Définition de données (LDD)

Primary keys

Exemple

```
CREATE TABLE supplier (
supplier_id numeric(10) not null,
supplier_name varchar2(50) not null,
contact_name varchar2(50),
CONSTRAINT pk_supplier PRIMARY KEY
(supplier_id)
);
```

4 - Définition de données (LDD)

Primary keys

Exemple avec de clé p. sur 2 colonnes

```
CREATE TABLE supplier (
supplier_id numeric(10) not null,
supplier_name varchar2(50) not null,
contact_name varchar2(50),
CONSTRAINT pk_supplier PRIMARY KEY
(supplier_id,supplier_name)
);
```

4 - Définition de données (LDD)

Primary keys

Pour créer la clé primaire après création de la table

Syntaxe:

```
ALTER TABLE table_name
ADD CONSTRAINT constraint_name PRIMARY KEY
(column1, column2, ... column_n);
```

Exemple :

```
ALTER TABLE supplier ADD CONSTRAINT
supplier_pk PRIMARY KEY (supplier_id);
```

4 - Définition de données (LDD)

Primary keys

<u>Suppression</u> . on utilise ALTER TABLE

Syntaxe :

```
ALTER TABLE table_name  DROP CONSTRAINT
constraint_name;
```

Exemple:

ALTER TABLE supplier DROP CONSTRAINT pk_supplier;

4 - Définition de données (LDD)

Primary keys

Désactiver/activer

On peut désactiver une clé primaire sans la supprimer avec le mot **DISABLE**

Syntaxe:

ALTER TABLE table_name **DISABLE** CONSTRAINT constraint_name;

Exemple :

ALTER TABLE supplier

 DISABLE CONSTRAINT pk_supplier;

4 - Définition de données (LDD)

Primary keys

Désactiver/activer

On peut activer une clé primaire désactivée avec le mot **ENABLE**

Syntaxe:

ALTER TABLE table_name **ENABLE** CONSTRAINT constraint_name;

Exemple :

ALTER TABLE supplier

ENABLE CONSTRAINT pk_supplier;

4 - Définition de données (LDD)

FOREIGN KEYS

Une clé etrangère d'une table (fille) référence une clé primaire d'une table (parent)

Syntaxe:

```
CREATE TABLE table_name (
column1 datatype null/not null,
column2 datatype null/not null,
...
CONSTRAINT fk_column FOREIGN KEY
(column1, column2, ... column_n)
REFERENCES parent_table (column1,
column2, ... column_n) );
```

FOREIGN KEYS

```
CREATE TABLE supplier (
 supplier_id number(10) not null,
 supplier_name varchar2(50) not null,
contact_name varchar2(50),
CONSTRAINT supplier_pk PRIMARY KEY (supplier_id)
 );

CREATE TABLE products (
 product_id number(10) not null,
supplier_id number(10) not null,
 CONSTRAINT fk_supplier FOREIGN KEY (supplier_id)
REFERENCES supplier (supplier_id)
 );
```

4 - Définition de données (LDD)

FOREIGN KEYS

La clé étrangère est définie après création de la table

Syntaxe

```
ALTER TABLE table_name  ADD
CONSTRAINT constraint_name FOREIGN
KEY (column1, column2, ... column_n)
REFERENCES parent_table (column1,
column2, ... column_n);
```

4 - Définition de données (LDD)

FOREIGN KEYS

Exemple:

ALTER TABLE products ADD CONSTRAINT fk_supplier FOREIGN KEY (supplier_id, supplier_name) REFERENCES supplier(supplier_id, supplier_name);

4 - Définition de données (LDD)

FOREIGN KEYS

Clé étrangère avec option DELECTE CASCADE

Par défaut la suppression d'une ligne sur la table parent n'est pas possible car référencé par une autre table.

Une clé étrangère avec l'option CASCADE DELETE signifie que si une ligne de la table parent est supprimée alors la ligne correspondante de la table fille est automatiquement supprimée. On appelle **suppression en cascade.**

4 - Définition de données (LDD)

FOREIGN KEYS

Clé étrangère avec option DELECTE CASCADE

Syntaxe:

```
CREATE TABLE table_name (
column1 datatype null/not null, column2 datatype
null/not null,
...
CONSTRAINT fk_column FOREIGN KEY (column1,
column2, ... column_n) REFERENCES parent_table
(column1, column2, ... column_n) ON DELETE
CASCADE );
```

Ou

```
ALTER TABLE table_name ADD CONSTRAINT
constraint_name FOREIGN KEY (column1, column2,
... column_n) REFERENCES parent_table (column1,
column2, ... column_n) ON DELETE CASCADE;
```

4 - Définition de données (LDD)

FOREIGN KEYS

Clé étrangère avec l'option SET NULL ON DELETE

Une clé étrangère avec l'option **SET NULL ON DELETE** signifie que la suppression d'une ligne de la table parent (référencée) entraine que le champ correspondant dans la table fille sera mis à NULL la le ligne de la table fille n'est pas supprimer.

FOREIGN KEYS

Clé étrangère avec l'option SET NULL ON DELETE

```
CREATE TABLE table_name (
column1 datatype null/not null,
column2 datatype null/not null,
...
CONSTRAINT fk_column FOREIGN KEY (column1, column2, ... column_n) REFERENCES parent_table (column1, column2, ... column_n) ON DELETE SET NULL
);
```

4 - Définition de données (LDD)

Contrainte d'intégrité (check)

La contrainte check permet de s'assurer que les données saisies dans une colonne sont dans une plage de valeurs définie à l'avance.

Syntaxe

```
CREATE TABLE table_name (
column1 datatype null/not null,
column2 datatype null/not null,
...
CONSTRAINT constraint_name
CHECK(column_name condition)
[DISABLE]
);
```

Disable est optionnel, crée une contrainte désactivée

4 - Définition de données (LDD)

Contrainte d'intégrité (check)

Exemple

```
ALTER TABLE vins ADD CONSTRAINT
ck_degre CHECK (degree >= 0);
```

-ajouter un colonne et sa contrainte

```
ALTER TABLE clients ADD (genre
CHAR CONSTRAINT ck_genre CHECK
(genre in ('M', 'F'));
```

Contrainte d'intégrité (check)

- Pour supprimer une contrainte.

ALTER TABLE table_name **DROP CONSTRAINT constraint_name**;

ALTER TABLE vins DROP CONSTRAINT **ck_degre;**

- Pour activer/desactiver une contraine

ALTER TABLE table_name ENABLE CONSTRAINT constraint_name;

ALTER TABLE table_name DISABLE CONSTRAINT constraint_name;

4 - Définition de données (LDD)

Contrainte d'intégrité (check)

Exemple

- Pour activer/desactiver une contraine

ALTER TABLE vins ENABLE CONSTRAINT ck_degree;

ALTER TABLE vins DISABLE CONSTRAINT ck_degree;

5 – Contrôle des données

Gestion des privilèges

On peut accorder différents types et différents niveaux de privilèges aux utilisateurs sur les tables (objets).

Ces privilèges peuvent être toute combinaison de : SELECT, INSERT, UPDATE, DELETE, REFERENCES, ALTER, INDEX, or ALL.

5 – Contrôle des données

Gestion des privilèges

La syntaxe pour accorder des privilèges est:

```
GRANT privileges ON objet TO
utilisateur;
```

Objet: le nom de l'objet de la base de données sur lequel on veut accorder des privilèges.

Utilisateur: le nom de l'utilisateur qui bénéficiera des privilèges.

5 – Contrôle des données

Grant

Exemple

```
GRANT SELECT, INSERT, UPDATE,
DELETE ON vins TO Issa;
```

- Les privileges select, insert, update et delete sur la table vins sont accordés à l'utilisateur Issa.

```
GRANT ALL ON clients TO ISSA
```

- Tous les privilèges sont accordés à ISSA

5 – Contrôle des données

Grant

Exemple

```
GRANT ALL ON clients FROM public;
```

Le mot-clé **public** permet d’attribuer des privilèges à tous les utilisateurs.

5 – Contrôle des données

Revoke

De façon manière inverse, on peut révoquer tout ou une partie des privilèges accordées à un utilisateur avec le mot-clé **revoke**.

```
REVOKE privileges ON objet FROM
utilisateur;
```

5 – Contrôle des données

Revoke

Exemple:

```
REVOKE DELETE ON clients FROM Issa;
REVOKE ALL ON clients FROM public;
```

SPQlus

1- Connexion au compte utilisateur

→ Connect *utilisateu*

→ Password: *xxxxxx*

2- Afficher les définitions des colonnes d'une table

→ describe nom_table;

SPQlus

3- Afficher toutes les tables:

Select TABLE_NAME from USER_TABLES;

Exécuter un fichier sql (contenant des instructions sql) avec @

- @*nom_fichier.sql* (répertoire courant)
- @*chemin_fichier nom_fichier.sql* *(autre répertoire)*

SPQlus

4- Exécuter un fichier sql (contenant des instructions sql) avec START

Start *nom_fichier.sql* (répertoire courant)

Start chemin_fichier nom_fichier.sql (autre répertoire)

sql> start C:\Users\USER\Desktop\myscript.sql

SPQlus

5- Enregistrer les commandes sql exécutées dans slqplus dans un fichier

save *filename* [create|append]

Append: ajoute dans le filename*,*

Create : crée un nouveau filename

Programmation Oracle

- Procédure stocké

- PL/SQL (Procedural langage/SQL) est une extension procédural de SQL en langage de programmation
- Les manipulation des données et réquêtes SQL sont incluses dans du code procédural

4 Programmation Oracle

- Avantages de PL/SQL

4 Programmation Oracle

Structure

Comme dans tout langage procédural, le code est formé de blocs. Chaque bloc comprend une section de déclaration de variables, et un ensemble d’instruction.

4 Programmation Oracle

Structure

```
DECLARE
   /*déclaration de variables:  optionnelle */
BEGIN
 /*instructions  sql et pl/sql  à exécuter */

EXCEPTION
        /*Actions à faire en cas d'erreur: optionnel*/
END;
```

4 Programmation Oracle

Variable

Une variable se déclare comme suit:

```
nom type := [initialisation];
```

L'initialisation est optionnelle. Les types utilisés sont les mêmes que dans les tables.

4 Programmation Oracle

Variable

Exemple

```
DECLARE
c  varchar2 (15) := ' Hello World ! ' ;
BEGIN
  DBMSOUTPUT.PUT LINE( c ) ;
END;
/
```

Un script écrit en PL/SQL se termine obligatoirement par un /, sinon SQL+ ne l'interprète pas.

4 Programmation Oracle

Affichage

Pour afficher le contenu d'une variable, les procédures DBMS_OUTPUT.PUT() et DBMS_OUTPUT.PUT_LINE() prennent en argument une valeur à afficher ou une variable.

Dans SQL+, on peut activer ou désactiver l’affichage avec la commande

SET SERVEROUTPUT ON (active)

SET SERVEROUTPUT OFF (désactive)

4 Programmation Oracle

Traitement conditionnel

```
IF    /*condition * /
THEN
    /* instructions1* /
ELSE
    /*instructions2 */
END IF ;
```

4 Programmation Oracle

Traitements répétitifs ou itération

```
LOOP
    /*Instructions*/
END LOOP
```

On peut quitter la boucle à condition

```
LOOP
    /*instructions*/
EXIT WHEN /*condition*/ ;
END LOOP;
```

4 Programmation Oracle

Traitements répétitifs ou itération

La boucle FOR existe aussi en PL/SQL

FOR */*variable*/* **IN** */* debut*/* **..** */*fin*/*

LOOP

*/*instructions*/*

END LOOP;

4 Programmation Oracle

Traitements répétitifs ou itération

La boucle while aussi

```
WHILE /*Condition*/
 LOOP
        /*instructions*/
END LOOP;
```

4 Programmation Oracle

Tableau

Les types tableau doivent être définis explicitement par une déclaration de la forme:

TYPE */*nomype*/* **IS VARRAY (**/*taille*/ **) OF** /* *type Elements*/* **;**

- Type: nom du type tableau;
- Taille : le nombre maximal d'éléments
- TypeElements : type des éléments qui vont être stockés dans le tableau.

```
Type numTab IS VARRAY (10) of NUMBER;
```

4 Programmation Oracle

Tableau

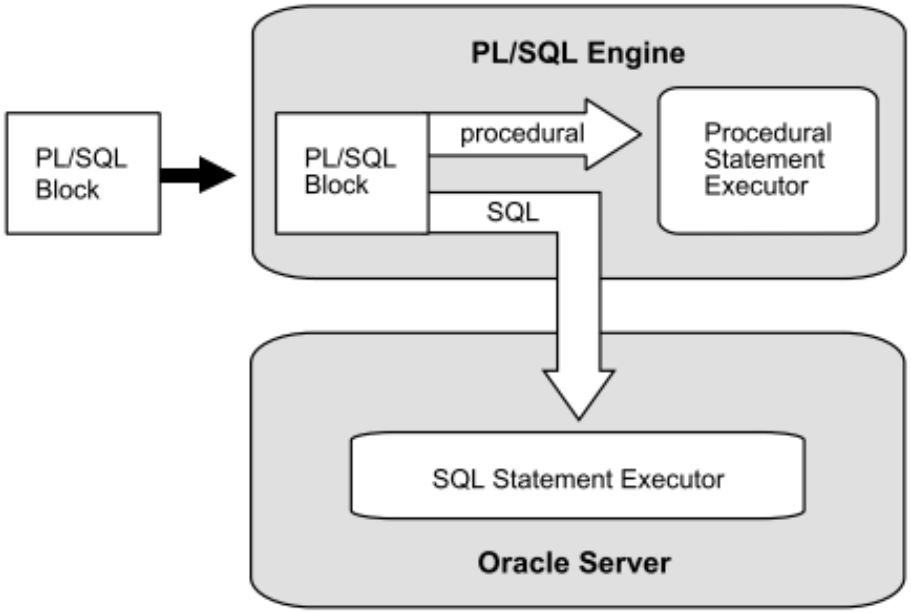
```
DECLARE
        TYPE numTab IS VARRAY (10) OF NUMBER;
        t numTab ;
BEGIN
        t = numTab();
        /*instructions* /
END;
/
```

Le type d’un tableau peut être utilisé au même titre que number ou varchar2.

4 Programmation Oracle

Interaction avec BD

Les scripts PL/SQL permet d'interagir avec une base de données.



4 Programmation Oracle

Interaction avec BD

INTO

On place dans une variable le résultat d'une réquête en utilisant **INTO.**

```
SELECT champ1, ..., champn  INTO  v1,
..., vn
FROM . . .
```

Affect e aux variable v_1, … v_n les valeurs retournées par la requête.

4 Programmation Oracle

Interaction avec BD

INTO

```
DECLARE
        nomclt varchar2(30);
        numcmd number(6) := 345 ;
BEGIN
        SELECT nom INTO nomclt
        FROM commandes clients
        WHERE  ncde = numcmd and ncli=num;
     DBMSOUTPUT.PUT_LINE( 'L a commande '  ||
numcmd || ‘est passée par  le client ’ || nomclt) ;
END;
/
```

4 Programmation Oracle

Interaction avec BD

%type

il est possible de référer directement au type d'une colonne avec le **%type** :

Syntaxe

`nomTable.nomColonne%type`

Exemple

nomclt **clients.nom%type ;**

numcmd **commandes.ncde%type := 345 ;**

4 Programmation Oracle

Interaction avec BD

%rowtype

Il est même possible de déclarer une structure pour représenter une ligne d'une table, le type porte alors le nom suivant : **nomTable%rowtype.**

4 Programmation Oracle

Transaction

Le système des transactions est l’un des mécanismes les plus puissants des SGBD récents.

Une transaction est un ensemble d'opérations « atomiques », c'est-a-dire indivisible.

4 Programmation Oracle

Transaction

On considère qu'un ensemble d'opérations est **indivisible** si une **exécution partielle** de ces instructions poserait des problèmes d'**intégrité** dans la base de données.

Par exemple, dans le cas d'une base de donnees de gestion de comptes en banque, un virement d'un compte a un autre se fait en deux temps. Débiter et créditer.

4 Programmation Oracle

Transaction

Les instructions COMMIT et ROLLBACK permet d'assurer l'exécution d'un ensemble d'opérations

```
/*Instruction*/
IF /*Erreur*/ THEN
        ROLLBACK;
ELSE
        COMMIT;
END;
```

ROLLBACK annule toutes les modifications faites depuis le début de la transaction (depuis le précèdent COMMIT),
COMMIT les enregistre définitivement dans la base de données.

4 Programmation Oracle

Transaction

La variable d'environnement AUTOCOMMIT, permet d'activer la gestion des transactions.

AUTOCOMMIT ON: chaque instruction a des répercussions immédiates dans la base.

AUTOCOMMIT OFF: les modifications ne sont effectives qu'une fois qu'un COMMIT a été exécute.

4 Programmation Oracle

Un sous-programme PL/SQL (subprogram) est un bloc PL/SQL nommée qui peut être appelée plusieurs fois. Si le sous programme prend des paramètres, leurs valeurs peuvent être différentes à chaque invocation.

Un subprogram est une procédure ou une fonction.

4 Programmation Oracle

Procedure

On définit une procédure de la façon suivante:

CREATE [OR REPLACE] PROCEDURE proc_name

[(parameter [,parameter])] **IS**

*/*Declaration de variable!*/*

BEGIN

*/*Instruction*/*

END;

4 Programmation Oracle

Procedure

Exemple

```
CREATE OR REPLACE PROCEDURE compteARebours (n NUMBER) IS
BEGIN
        IF n >= 0 THEN
        DBMSOUTPUT.PUT LINE(n ) ;
        compteARebours (n - 1 ) ;
        END IF ;
END;
```

4 Programmation Oracle

Procedure

On appelle de la procédure avec son nom.

Dans sql+, on utilise le mot-clé call.

```
SQL> call compteARebours(20)
```

Supprimer

DROP PROCEDURE *procedure_name*;

4 Programmation Oracle

Fonction

On définit une fonction de la façon suivante:

CREATE [OR REPLACE] FUNCTION *nom_funct* [(parameter [,parameter])]

RETURN type_retourne **IS** |

/*Déclaration de variable*/

BEGIN

*/*instructions*/*

END [function_name];

4 Programmation Oracle

Fonction

Exemple

```
CREATE OR REPLACE FUNCTION module ( a NUMBER, b NUMBER) RETURN NUMBER  IS
BEGIN
        IF a < b THEN
                RETURN a ;
        ELSE
                RETURN module ( a - b , b ) ;
        END IF;
END;
```

4 Programmation Oracle

Fonction

On appelle les fonctions en PL/SQL de la même manière que les procédures.

En SQL+, on passe par une pseudo-table nommée DUAL de la façon suivante:

SELECT module (21,12) FROM **DUAL;**

4 Programmation Oracle

Triggers ou déclencheurs

Principe

Un trigger est comme une procédure stockée dans la base, déclenché lors de l'occurrence d'un événement particulier.

La plupart du temps les triggers sont déclenchés par la modification du contenu d'une table.

4 Programmation Oracle

Package

Principe

Un package est un module de programmes incluant procédures et / ou fonctions fonctionnellement dépendantes.
Un package est composé de 2 parties :

- la spécification (introduite par '**CREATE PACKAGE**') liste les entêtes de procédures et fonctions contenues dans le package,
- le corps du package (introduit par '**CREATE PACKAGE BODY**') qui contient le code effectif des procédures et fonctions déclarées précédemment.

Considérez les Tables de Données suivantes:

Table Avion :

Numav	Capacite	Type	Entrepot
14	25	A400	Garches
345	75	B200	Maubeuge

Table Pilote :

Matricule	Nom	Ville	Age	Salaire
1	Figue	Cannes	45	28004
2	Lavande	Touquet	24	11758

Table Passager :

Numab	Nomab
1	Nifance
8	Téarice

Table Vol :

Numvol	Heure_depart	Heure_arrivee	Ville_depart	Ville_arrivee
AL12	08-18	09-12	Paris	Lille
AF8	11-20	23-54	Vaux	Rio

Table Depart :

Numvol	Date_dep	Numav	Matricule
AF118	31-12-95	14	1
AF212	19-12-95	345	2

Table Reservation :

Numab	Numvol	Date_dep
1	AF118	31-12-95
8	AF118	31-12-95

Exercice 1

Ecrivez un programme PL/SQL qui insère le vol AF110 partant de Paris à 21h40 et arrivant à Dublin à 23h10 (hypothèse : le vol n'est pas déjà présent dans la table).

Exercice 2

Ecrivez un programme PL/SQL qui propose les vols pour un tour du monde au départ de Paris avec des escales et des durées d'escale prédéfinies dans la table Escales. Le nombre d'escales à faire doit être demandé à l'utilisateur. Hypothèse de travail : pour chaque escale il existe un vol et un seul satisfaisant les contraintes.

Escales :

Numescale	Ville_escale	Duree_escale
1	Moscou	5
2	Singapour	5
3	Sydney	4
4	Tahiti	4
5	Honolulu	4
6	Los Angeles	5
7	New York	4
8	Londres	3

Exercice 3

Ecrivez un programme PL/SQL qui calcule la moyenne des salaires des pilotes dont l'âge est entre 30 et 40 ans.

Exercice 4

Modifiez le programme PL/SQL de l'exercice 2 pour qu'il fonctionne même si plusieurs vols satisfont les contraintes.

Exercice 5

Ecrivez une procédure PL/SQL capable de faire des propositions de tours du monde, prenant en entrée la ville de départ (qui est aussi la destination finale) et deux bornes (supérieure et inférieure) pour le nombre d'escales. Dans ce cas il n'y a pas de liste prédéfinie d'escales et on ne s'intéresse pas à la durée des escales. Cherchez à utiliser une procédure ou fonction récursive (PL/SQL admet la récursivité).

Exercice 6

Transformez le programme du dernier exercice du précédent ED en procédure prenant comme paramètre le nombre d'escales. Regroupez-la avec la procédure de l'exercice précédent dans un paquetage. Pouvons-nous définir des curseurs et/ou variables communes entre les deux procédures ?